



Grafs III

Algorísmica Avançada | Enginyeria Informàtica

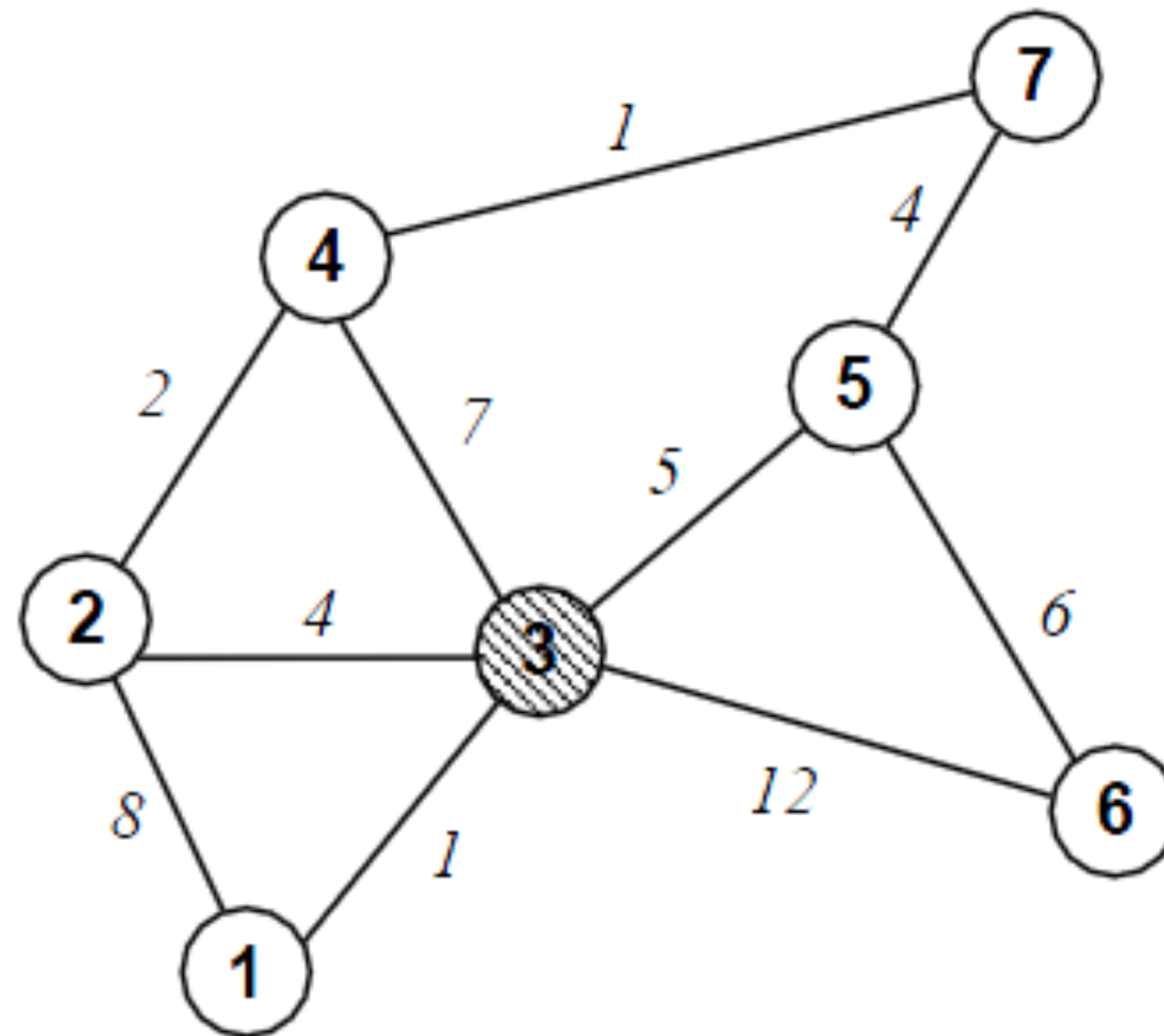
Santi Seguí | 2026-2027

Sessió

- 1. Wrap-up: Dijkstra
- 2. Bellman Ford
 - Trobar camí més curt amb grafs amb pesos
- 3. Floyd Warshall
- 3. Camí/Circuit Eulerià

Dijkstra

- Algorithme de **Dijkstra**: exemple graf **NO** dirigé



Dijkstra

```
1  Dijkstra(G, s):
2      per a cada vèrtex v de G:
3          dist[v] ← ∞
4          prev[v] ← NUL
5          visitat[v] ← fals
6      dist[s] ← 0
7
8      repeteix |V| vegades:
9          u ← vèrtex no visitat amb dist[u] mínima
10         si dist[u] = ∞: atura
11         visitat[u] ← cert
12
13         per a cada veí v de u amb pes w(u, v):
14             si no visitat[v]:
15                 alt ← dist[u] + w(u, v)
16                 si alt < dist[v]:
17                     dist[v] ← alt
18                     prev[v] ← u
19
20     retorna dist, prev
```

Dijkstra (amb HEAP)

```
1  Dijkstra(G, s):  
2      per a cada vèrtex v de G:  
3          dist[v] ← ∞  
4          prev[v] ← NUL  
5      dist[s] ← 0  
6  
7      Q ← cua de prioritat amb tots els vèrtexs (clau = dist)  
8  
9      mentre Q no sigui buida:  
10         u ← Q.extreu_mínim()  
11  
12         per a cada veí v de u amb pes w(u, v):  
13             alt ← dist[u] + w(u, v)  
14             si alt < dist[v]:  
15                 dist[v] ← alt  
16                 prev[v] ← u  
17                 Q.decrementa_clau(v, alt)  
18  
19     retorna dist, prev
```

Dijkstra (amb HEAP)

Pensem la complexitat

```
1  Dijkstra(G, s):  
2      per a cada vèrtex v de G:  
3          dist[v] ← ∞  
4          prev[v] ← NUL  
5      dist[s] ← 0  
6  
7      Q ← cua de prioritat amb tots els vèrtexs (clau = dist)  
8  
9      mentre Q no sigui buida:  
10         u ← Q.extreu_mínim()  
11  
12         per a cada veí v de u amb pes w(u, v):  
13             alt ← dist[u] + w(u, v)  
14             si alt < dist[v]:  
15                 dist[v] ← alt  
16                 prev[v] ← u  
17                 Q.decrementa_clau(v, alt)  
18  
19     retorna dist, prev
```

Dijkstra (amb cua de prioritat)

```
1  Dijkstra(G, s):
2      per a cada vèrtex v de G:
3          dist[v] ← ∞
4          prev[v] ← NUL
5      dist[s] ← 0
6
7      Q ← cua de prioritat buida
8      Q.insereix(s, 0)
9
10     mentre Q no sigui buida:
11         u ← Q.extreu_mínim()
12         si u ja està visitat: continua
13         marca u com a visitat
14
15         per a cada veí v de u amb pes w(u, v):
16             alt ← dist[u] + w(u, v)
17             si alt < dist[v]:
18                 dist[v] ← alt
19                 prev[v] ← u
20                 Q.insereix(v, alt)
21
22     retorna dist, prev
```

Dijkstra

- Complexitat
 - Per a qualsevol estructura de dades, la complexitat de l'algoritme:

$$O(|E| \cdot T_{dk} + |V| \cdot T_{em}),$$

on T_{dk} és la complexitat de la funció *decrease-key* i T_{em} la complexitat de la funció *extract-minimum*.

Implementation	extractMin	Insert/decreasekey	Total
Array			
Binary Heap			

Dijkstra

- Complexitat
 - Per a qualsevol estructura de dades, la complexitat de l'algoritme:

$$O(|E| \cdot T_{dk} + |V| \cdot T_{em}),$$

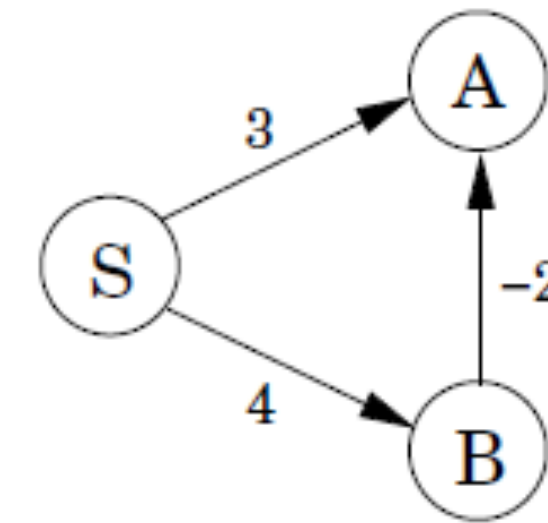
on T_{dk} és la complexitat de la funció *decrease-key* i T_{em} la complexitat de la funció *extract-minimum*.

Implementation	extractMin	Insert/decreasekey	Total
Array	$O(V)$	$O(1)$	$O(V ^2)$
Binary Heap	$O(\log V)$	$O(\log V)$	$O((V + E)\log(V))$

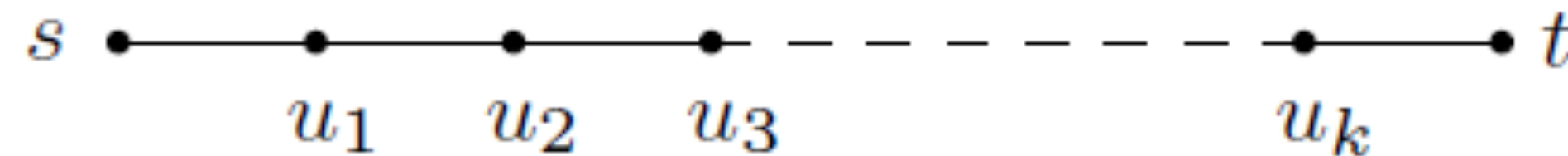
Dijkstra

- **Dijkstra** amb pesos negatius:

procedure update $((u, v) \in E)$
 $\text{dist}(v) = \min\{\text{dist}(v), \text{dist}(u) + l(u, v)\}$



- Amb Dijkstra sempre arribem de s a t amb camí mínim independentment de l'ordre dels pesos de les arestes si aquests són positius. **No amb negatius!**



Bellman-Ford

Bellman-Ford

- **Solució?** Canviem l'algorisme perquè es calculin totes les distàncies simultàniament.
→ actualitzar totes les arestes **$|V|-1$ vegades**

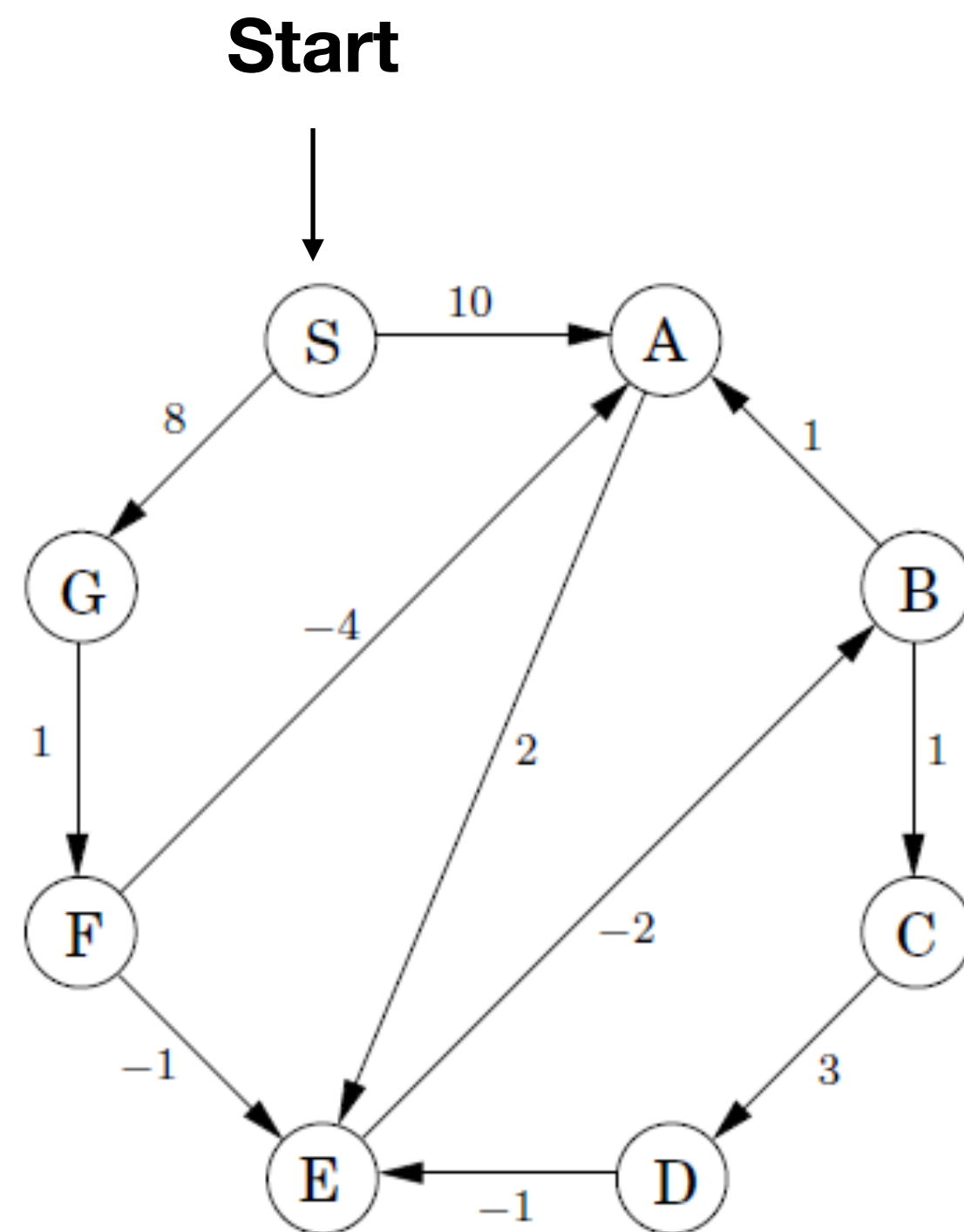
- **Bellman-Ford**

```
procedure shortest-paths( $G, l, s$ )  
Input:   Directed graph  $G = (V, E)$ ;  
         edge lengths  $\{l_e : e \in E\}$  with no negative cycles;  
         vertex  $s \in V$   
Output:  For all vertices  $u$  reachable from  $s$ ,  $\text{dist}(u)$  is set  
         to the distance from  $s$  to  $u$ .  
  
for all  $u \in V$ :  
     $\text{dist}(u) = \infty$   
     $\text{prev}(u) = \text{nil}$   
  
 $\text{dist}(s) = 0$   
repeat  $|V| - 1$  times:  
    for all  $e \in E$ :  
        update( $e$ )
```

```
procedure update( $(u, v) \in E$ )  
 $\text{dist}(v) = \min\{\text{dist}(v), \text{dist}(u) + l(u, v)\}$ 
```

Bellman-Ford

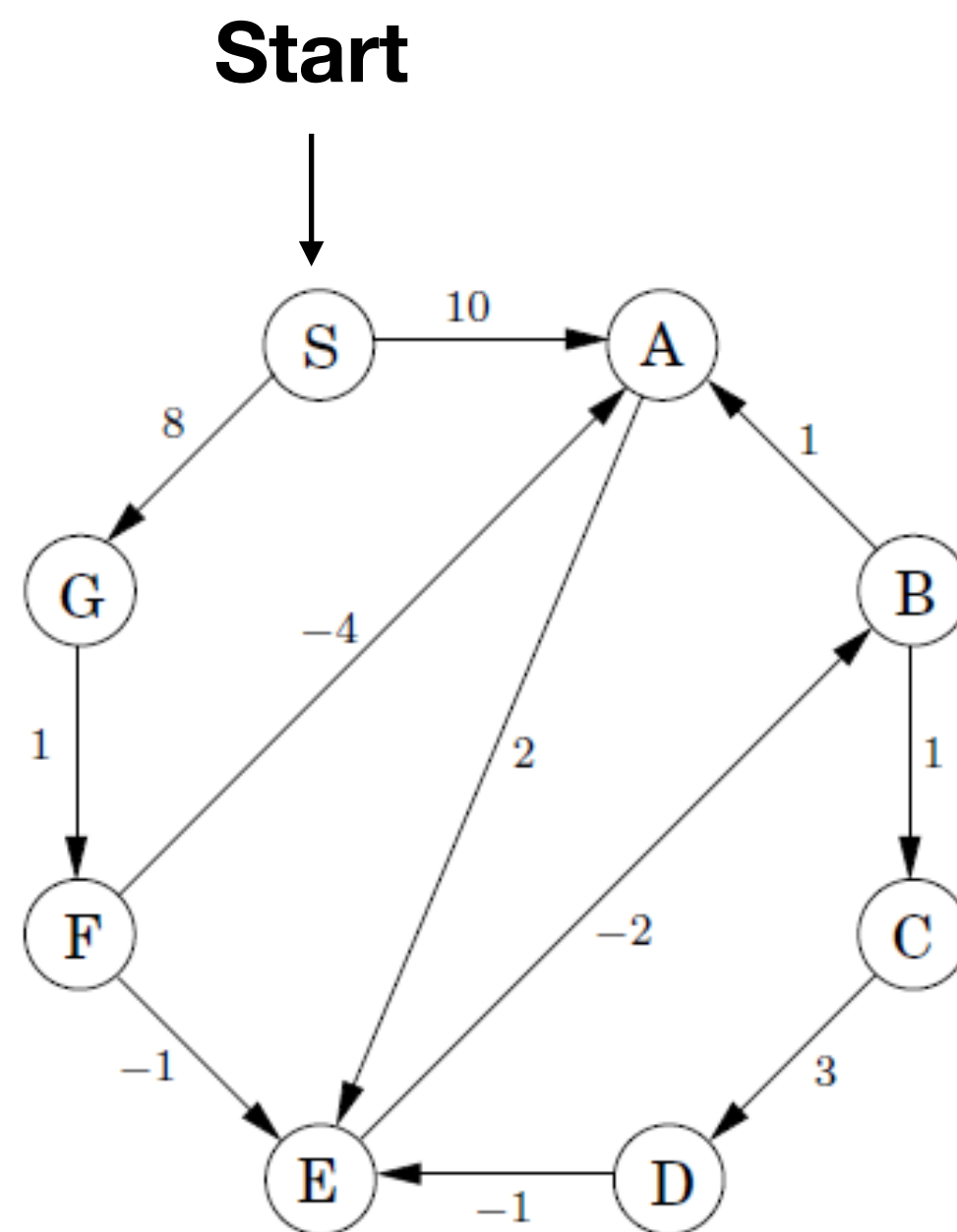
- Implementació: si en una iteració cap aresta e s'actualitza $\rightarrow$ finalitzar



	Iteration							
Node	0	1	2	3	4	5	6	7
S								
A								
B								
C								
D								
E								
F								
G								

Bellman-Ford

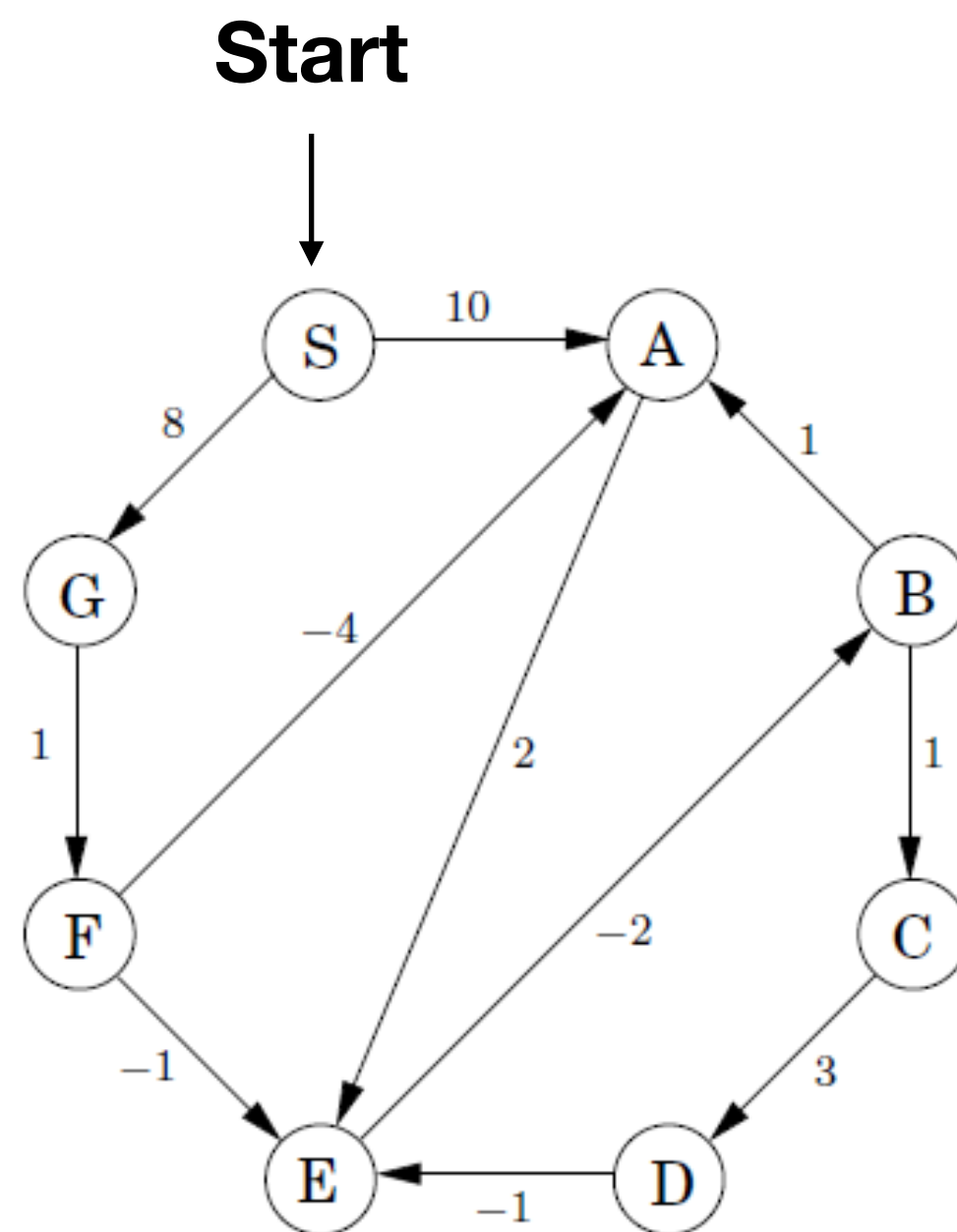
- Implementació: si en una iteració cap aresta e s'actualitza $\rightarrow$ finalitzar



Node	Iteration							
	0	1	2	3	4	5	6	7
S	0							
A	∞							
B	∞							
C	∞							
D	∞							
E	∞							
F	∞							
G	∞							

Bellman-Ford

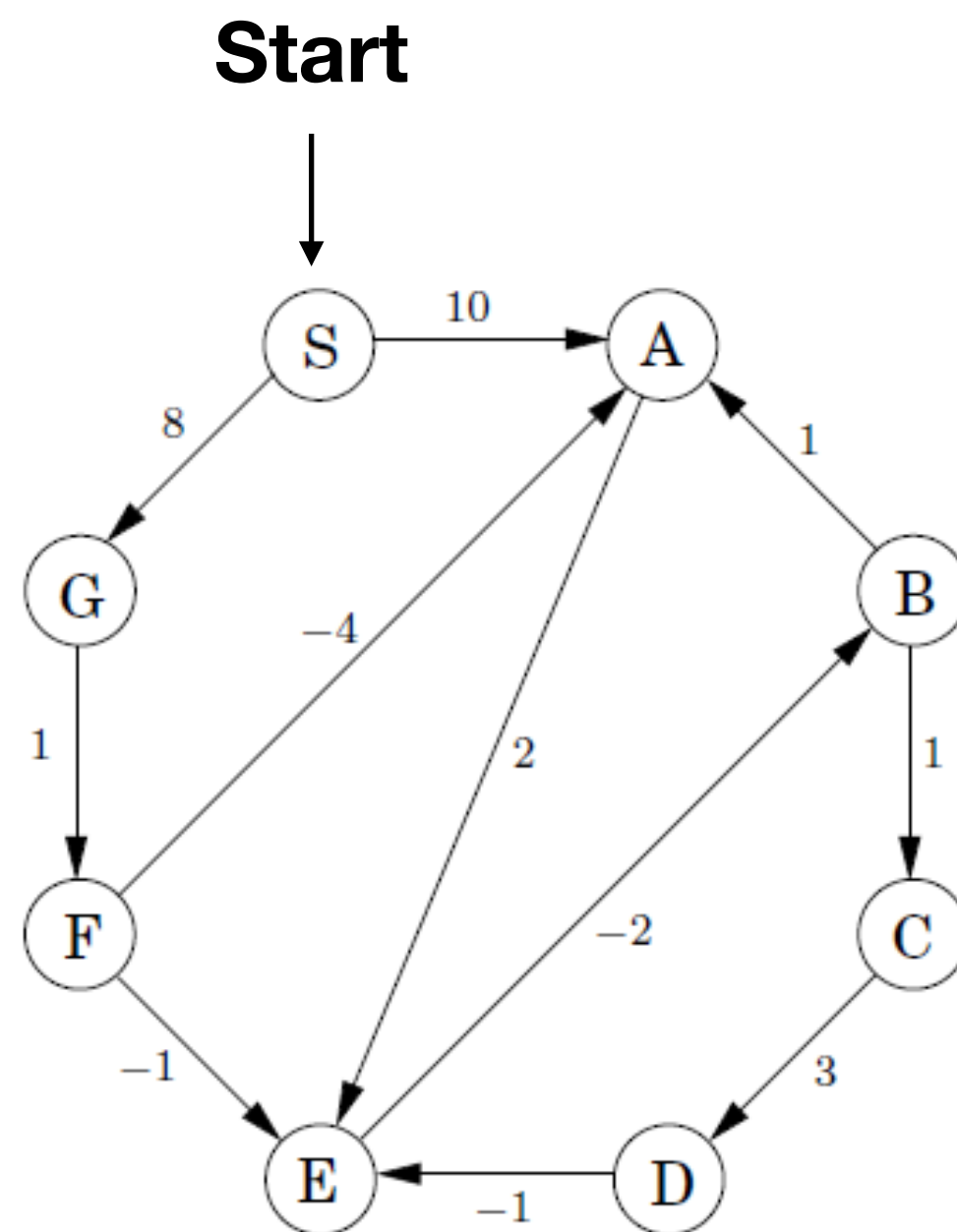
- Implementació: si en una iteració cap aresta e s'actualitza $\rightarrow$ finalitzar



Node	Iteration							
	0	1	2	3	4	5	6	7
S	0	0						
A	∞	10						
B	∞	∞						
C	∞	∞						
D	∞	∞						
E	∞	∞						
F	∞	∞						
G	∞	8						

Bellman-Ford

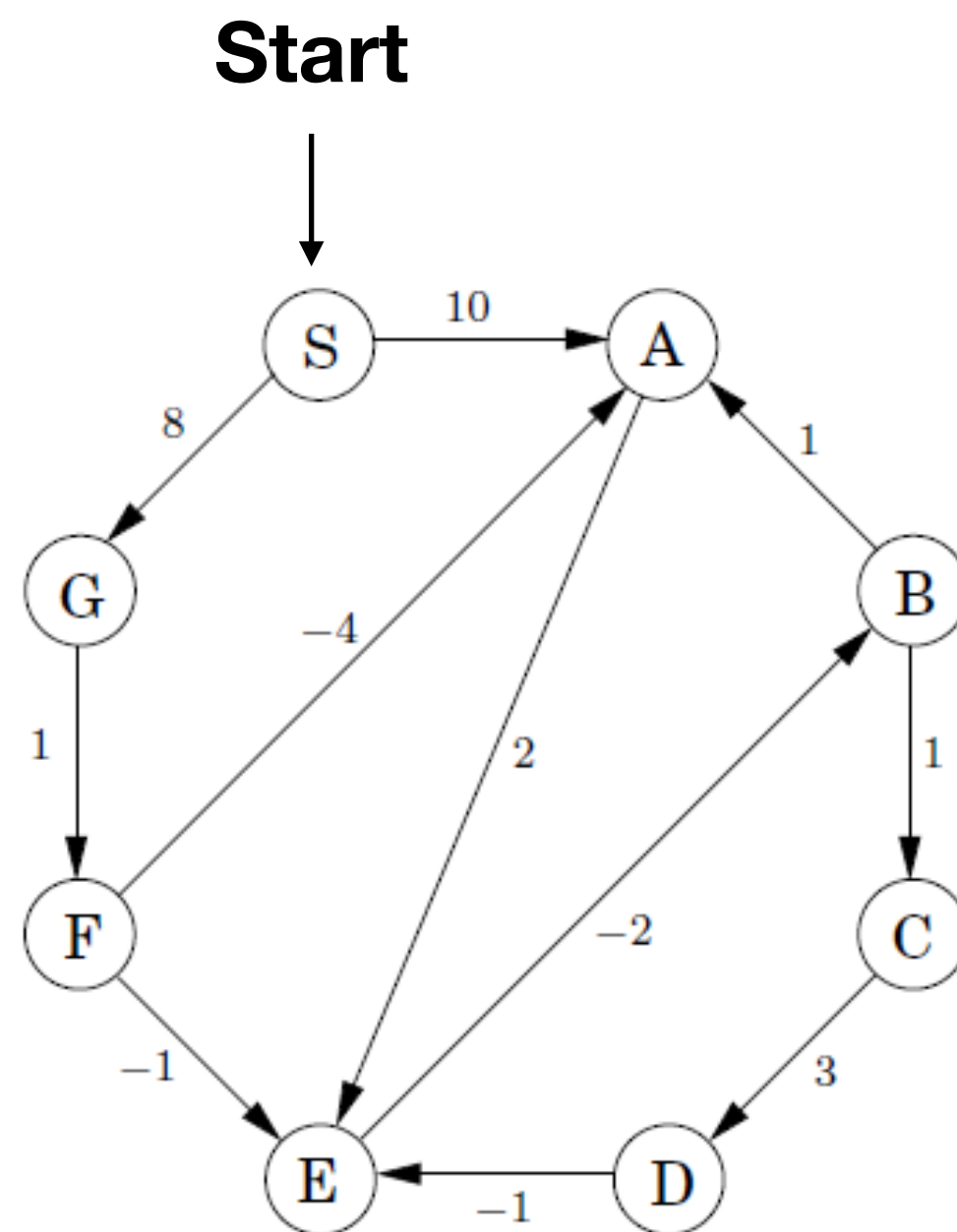
- Implementació: si en una iteració cap aresta e s'actualitza $\rightarrow$ finalitzar



Node	Iteration							
	0	1	2	3	4	5	6	7
S	0	0	0					
A	∞	10	10					
B	∞	∞	∞					
C	∞	∞	∞					
D	∞	∞	∞					
E	∞	∞	12					
F	∞	∞	9					
G	∞	8	8					

Bellman-Ford

- Implementació: si en una iteració cap aresta e s'actualitza $\rightarrow$ finalitzar



Node	Iteration							
	0	1	2	3	4	5	6	7
S	0	0	0	0				
A	∞	10	10	5				
B	∞	∞	∞	10				
C	∞	∞	∞	∞				
D	∞	∞	∞	∞				
E	∞	∞	12	8				
F	∞	∞	9	9				
G	∞	8	8	8				

Complexitat ?

```
procedure shortest-paths( $G, l, s$ )  
Input:   Directed graph  $G = (V, E)$ ;  
         edge lengths  $\{l_e : e \in E\}$  with no negative cycles;  
         vertex  $s \in V$   
Output:  For all vertices  $u$  reachable from  $s$ ,  $\text{dist}(u)$  is set  
         to the distance from  $s$  to  $u$ .  
  
for all  $u \in V$ :  
     $\text{dist}(u) = \infty$   
     $\text{prev}(u) = \text{nil}$   
  
 $\text{dist}(s) = 0$   
repeat  $|V| - 1$  times:  
    for all  $e \in E$ :  
        update( $e$ )
```

?

Complexitat

```
procedure shortest-paths( $G, l, s$ )  
Input:   Directed graph  $G = (V, E)$ ;  
        edge lengths  $\{l_e : e \in E\}$  with no negative cycles;  
        vertex  $s \in V$   
Output:  For all vertices  $u$  reachable from  $s$ ,  $\text{dist}(u)$  is set  
        to the distance from  $s$  to  $u$ .  
  
for all  $u \in V$ :  
     $\text{dist}(u) = \infty$   
     $\text{prev}(u) = \text{nil}$   
  
 $\text{dist}(s) = 0$   
repeat  $|V| - 1$  times:  
    for all  $e \in E$ :  
        update( $e$ )
```

$$O(|V| \cdot |E|)$$

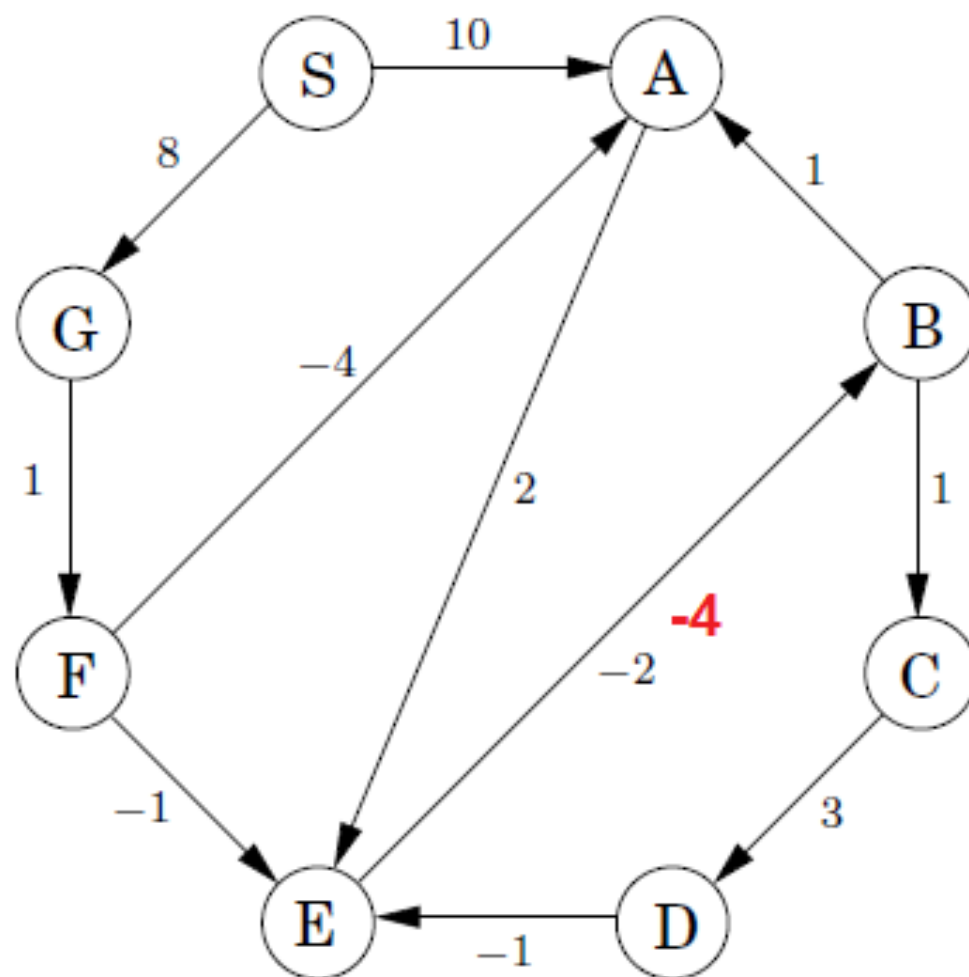
Problema:

Podem trobar **cicles amb cost negatiu** quan tenim arestes amb costos negatius.

Bellman-Ford

- **Cicles Negatius**

$A \rightarrow E \rightarrow B \rightarrow A$



Els podem trobar amb l'algorisme Bellman-Ford

El camí mínim té com a màxim longitud $|V|-1$

Podem detectar cicles si fem una iteració extra $|V|$

-> Hi ha cicle negatiu si a la iteració $|V|$ algun cost és actualitzat.

Floyd Warshall

All Short Paths

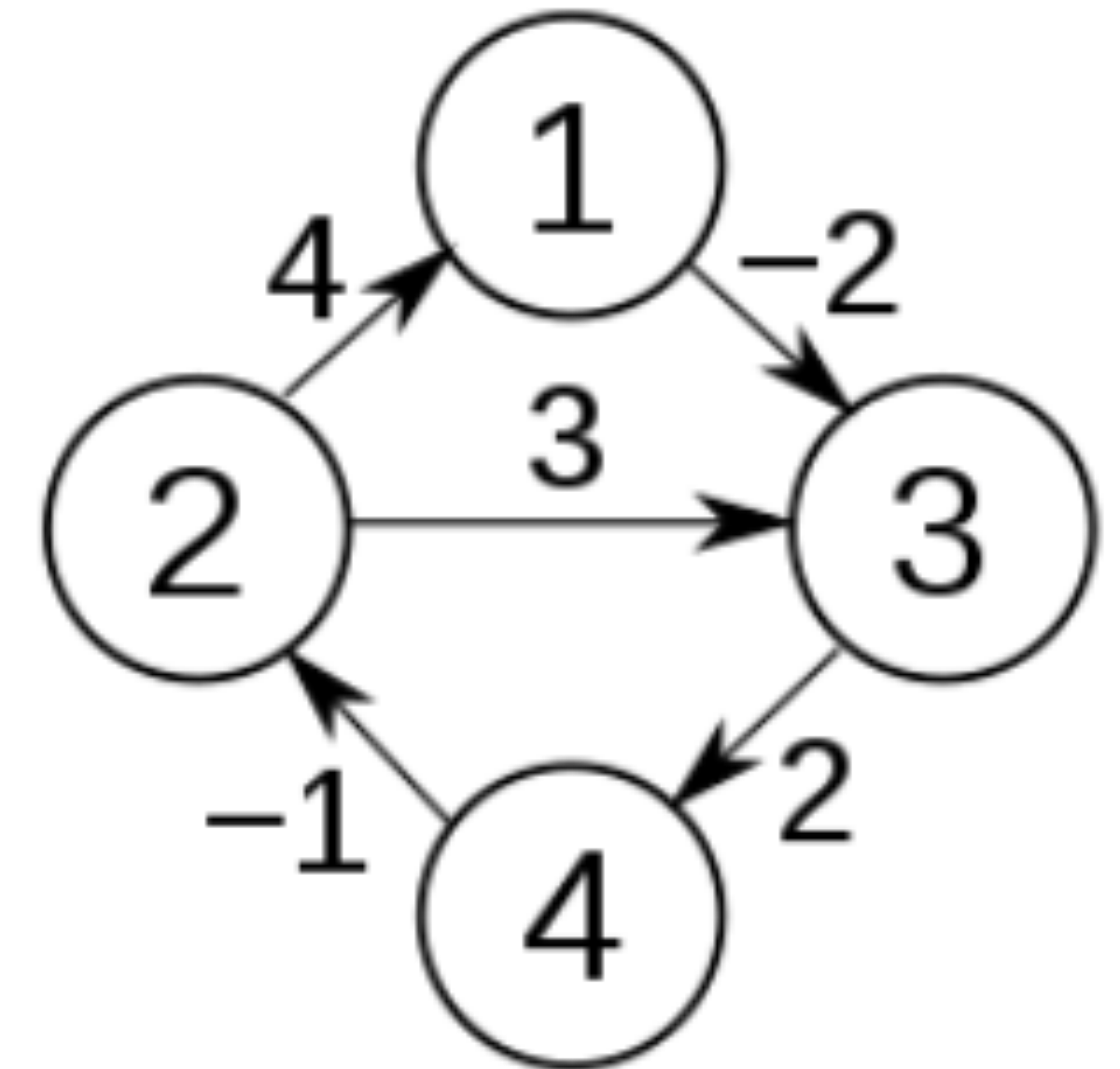
- Donat un graf **G** volem trobar el camí més curt entre totes les parelles dels seus nodes.
- Podem utilitzar Dijkstra per trobar tots els camins més curts?
 - Com ho fem?
- Quina serà la seva complexitat?

All Short Paths

- Donat un graf **G** volem trobar el camí més curt entre totes les parelles dels seus nodes.
- Podem utilitzar Dijkstra per trobar tots els camins més curts? **Sí!**
 - Com ho fem? **Executem Dijkstra per a cada node del nostre graf**
- Quina serà la seva complexitat? **La complexitat de Dijkstra mitjançant (cua prioritària) és $O((V + E)\log(V))$**
 - Per tant per trobar els camins mínims entre totes les parelles de nodes, la complexitat serà: $O((|V|^2 + |V| \cdot |E|)\log(V))$

All Short Paths - Floyd Warshall

```
1 let dist be a  $|V| \times |V|$  array of minimum distances initialized to  $\infty$  (infinity)
2 for each edge  $(u,v)$ 
3    $\text{dist}[u][v] \leftarrow w(u,v)$  // the weight of the edge  $(u,v)$ 
4 for each vertex  $v$ 
5    $\text{dist}[v][v] \leftarrow 0$ 
6 for  $k$  from 1 to  $|V|$ 
7   for  $i$  from 1 to  $|V|$ 
8     for  $j$  from 1 to  $|V|$ 
9       if  $\text{dist}[i][j] > \text{dist}[i][k] + \text{dist}[k][j]$ 
10          $\text{dist}[i][j] \leftarrow \text{dist}[i][k] + \text{dist}[k][j]$ 
11       end if
```

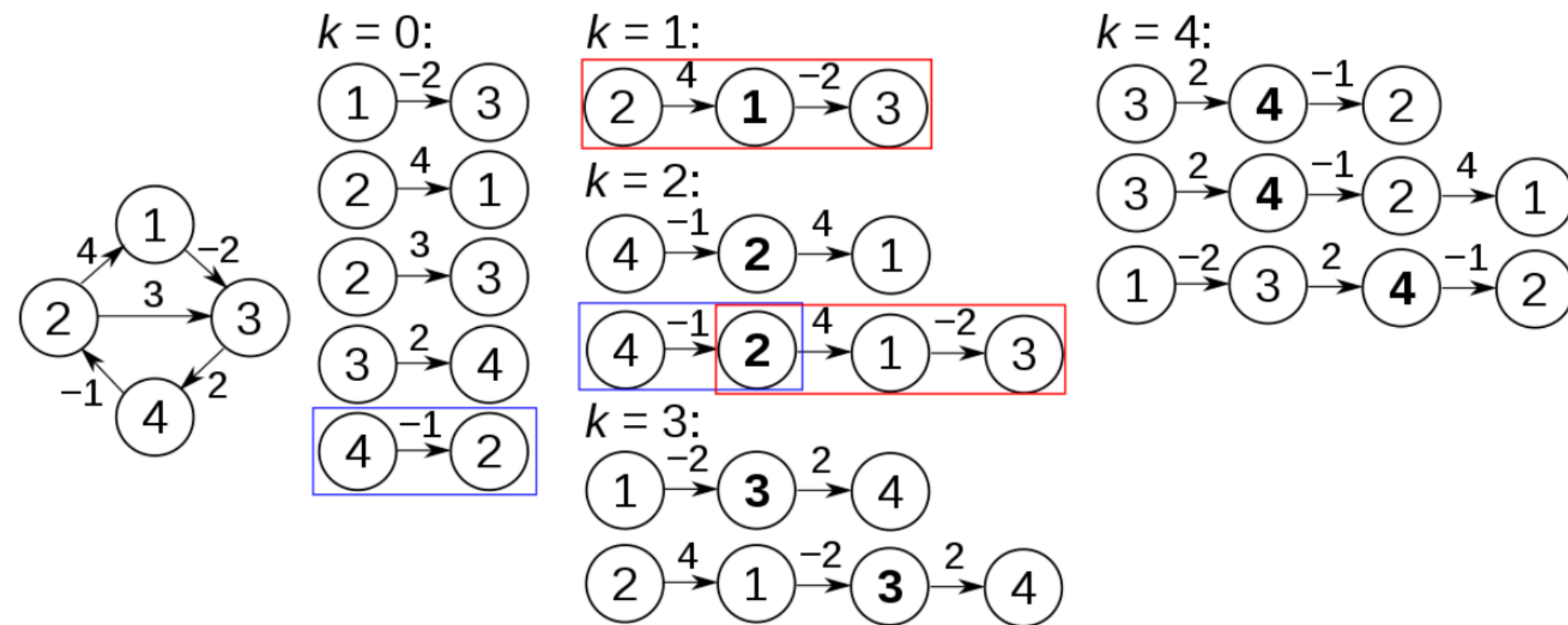


Trobeu la solució

Example [\[edit\]](#)

Exercici: All Short Paths

The algorithm above is executed on the graph on the left below:



Prior to the first recursion of the outer loop, labeled $k = 0$ above, the only known paths correspond to the single edges in the graph. At $k = 1$, paths that go through the vertex 1 are found: in particular, the path $[2, 1, 3]$ is found, which has fewer edges but is longer (in terms of weight). At $k = 2$, paths going through the vertices $\{1, 2\}$ are found. The red and blue boxes show how the path $[4, 2, 1, 3]$ is assembled from the two known paths $[4, 2]$ and $[2, 1, 3]$. The path $[4, 2, 3]$ is not considered, because $[2, 1, 3]$ is the shortest path encountered so far from 2 to 3. At $k = 3$, paths going through the vertices $\{1, 2, 3\}$ are found. Finally, at $k = 4$, all shortest paths are found.

The distance matrix at each iteration of k , with the updated distances in **bold**, will be:

$k = 0$		j				
		1	2	3	4	
i	1	0	∞	-2	∞	
	2	4	0	3	∞	
	3	∞	∞	0	2	
	4	∞	-1	∞	0	

$k = 1$		j				
		1	2	3	4	
i	1	0	∞	-2	∞	
	2	4	0	2	∞	
	3	∞	∞	0	2	
	4	∞	-1	∞	0	

$k = 2$		j				
		1	2	3	4	
i	1	0	∞	-2	∞	
	2	4	0	2	∞	
	3	∞	∞	0	2	
	4	3	-1	1	0	

$k = 3$		j				
		1	2	3	4	
i	1	0	∞	-2	0	
	2	4	0	2	4	
	3	∞	∞	0	2	
	4	3	-1	1	0	

$k = 4$		j				
		1	2	3	4	
i	1	0	-1	-2	0	
	2	4	0	2	4	
	3	5	1	0	2	
	4	3	-1	1	0	

All Short Paths - Floyd Warshall

- L'algorisme de **Floyd-Warshall** proporciona les longituds dels camins mínims entre tots els parells de vèrtexs. Però com obtenim els camins??
- Amb petites modificacions és possible crear un mètode per a reconstruir el camí real entre dos vèrtexs.

All Short Paths - Floyd Warshall

- L'algorisme de **Floyd-Warshall** proporciona les longituds dels camins mínims entre tots els parells de vèrtexs. Però com obtenim els camins??
- Amb petites modificacions és possible crear un mètode per a reconstruir el camí real entre dos vèrtexs.

```
let dist be a  $|V| \times |V|$  array of minimum distances initialized to  $\infty$  (infinity)
let next be a  $|V| \times |V|$  array of vertex indices initialized to null
```

```
procedure FloydWarshallWithPathReconstruction() is
  for each edge (u, v) do
    dist[u][v]  $\leftarrow$  w(u, v) // The weight of the edge (u, v)
    next[u][v]  $\leftarrow$  v
  for each vertex v do
    dist[v][v]  $\leftarrow$  0
    next[v][v]  $\leftarrow$  v
  for k from 1 to  $|V|$  do // standard Floyd-Warshall implementation
    for i from 1 to  $|V|$ 
      for j from 1 to  $|V|$ 
        if dist[i][j] > dist[i][k] + dist[k][j] then
          dist[i][j]  $\leftarrow$  dist[i][k] + dist[k][j]
          next[i][j]  $\leftarrow$  next[i][k]
```

```
procedure Path(u, v)
  if next[u][v] = null then
    return []
  path = [u]
  while u  $\neq$  v
    u  $\leftarrow$  next[u][v]
    path.append(u)
  return path
```

All Short Paths - Floyd Warshall

- L'algorisme de **Floyd-Warshall** proporciona les longituds dels camins mínims entre tots els parells de vèrtexs. Però com obtenim els camins??
- Amb petites modificacions és possible crear un mètode per a reconstruir el camí real entre dos vèrtexs.
- Quina és la complexitat en termes de temps?

```
let dist be a  $|V| \times |V|$  array of minimum distances initialized to  $\infty$  (infinity)
let next be a  $|V| \times |V|$  array of vertex indices initialized to null
```

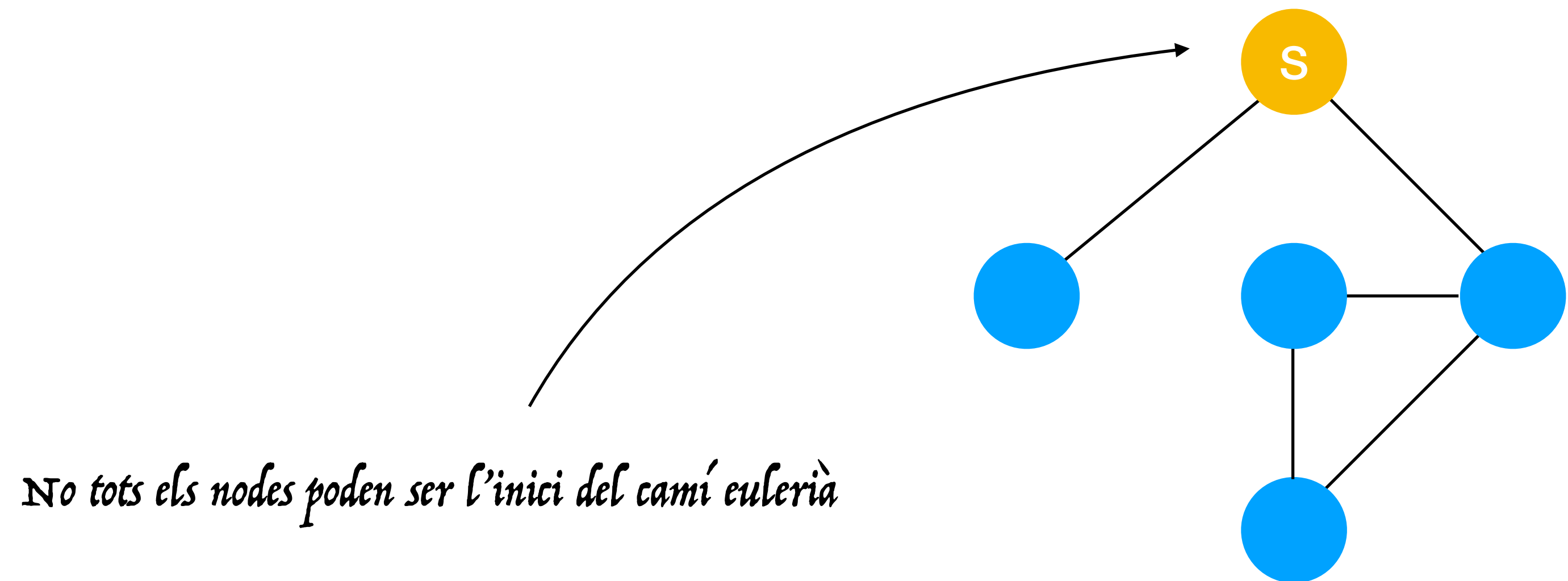
```
procedure FloydWarshallWithPathReconstruction() is
  for each edge (u, v) do
    dist[u][v]  $\leftarrow$  w(u, v) // The weight of the edge (u, v)
    next[u][v]  $\leftarrow$  v
  for each vertex v do
    dist[v][v]  $\leftarrow$  0
    next[v][v]  $\leftarrow$  v
  for k from 1 to |V| do // standard Floyd-Warshall implementation
    for i from 1 to |V|
      for j from 1 to |V|
        if dist[i][j] > dist[i][k] + dist[k][j] then
          dist[i][j]  $\leftarrow$  dist[i][k] + dist[k][j]
          next[i][j]  $\leftarrow$  next[i][k]
```

```
procedure Path(u, v)
  if next[u][v] = null then
    return []
  path = [u]
  while u  $\neq$  v
    u  $\leftarrow$  next[u][v]
    path.append(u)
  return path
```

Camí/Circuit Eulerià

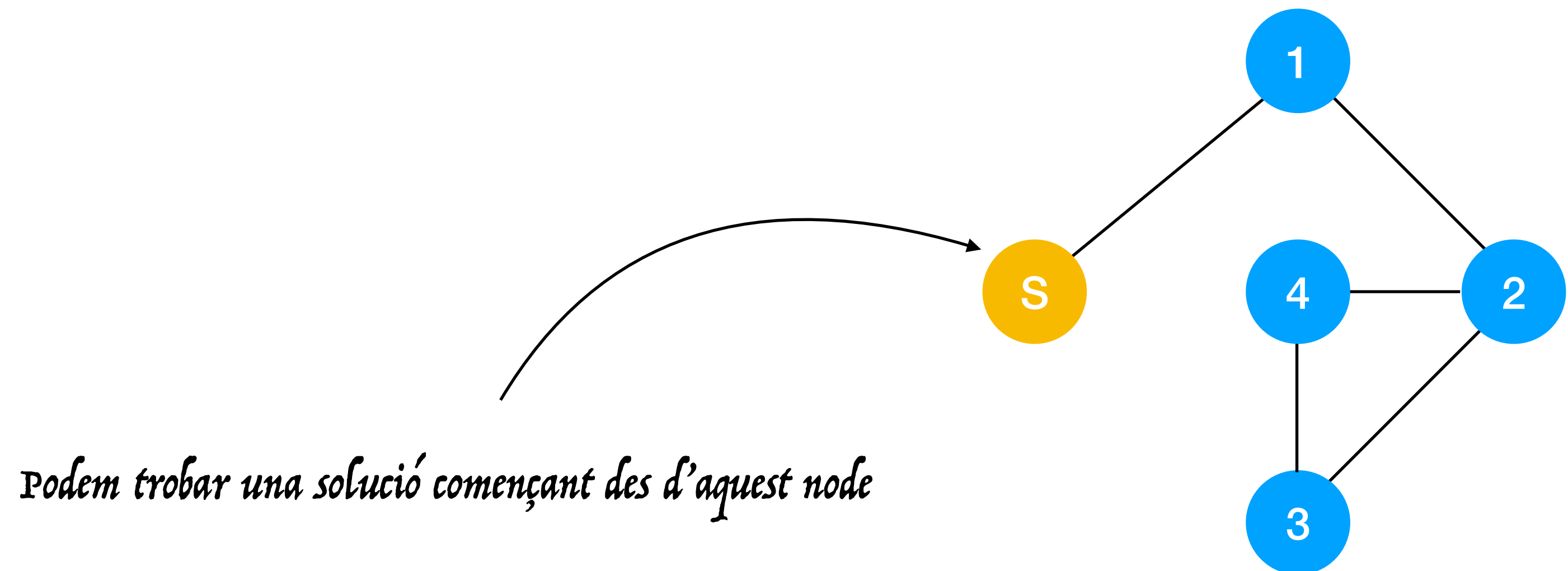
Camí Eulerià

- Un **camí Eulerià** és aquell camí que recorre **totes les arestes** d'un graf passant una i només una vegada per a cada una d'elles.
- No tots els grafs tenen un camí Eulerià



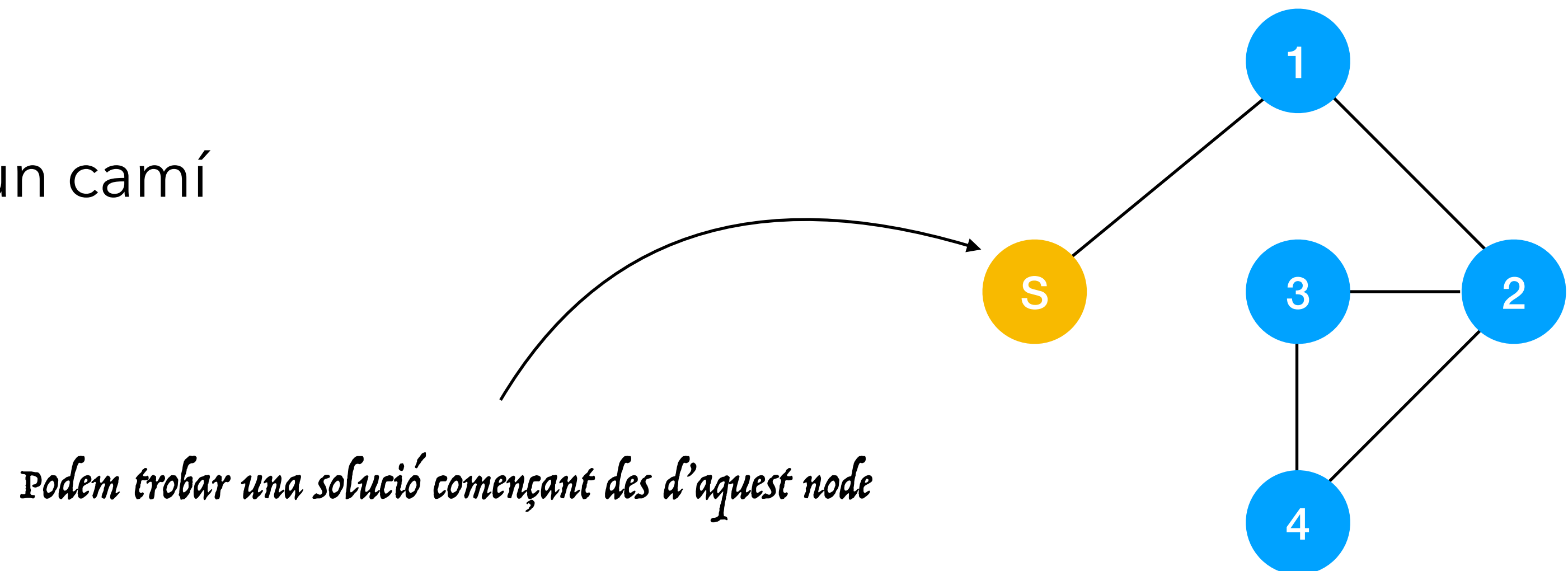
Camí Eulerià

- Un **camí Eulerià** és aquell camí que recorre **totes les arestes** d'un graf passant una i només una vegada per a cada una d'elles.
- No tots els grafs tenen un camí Eulerià
- El graf de la dreta té un camí Eulerià, però per trobar-lo hem de començar des de un node en concret



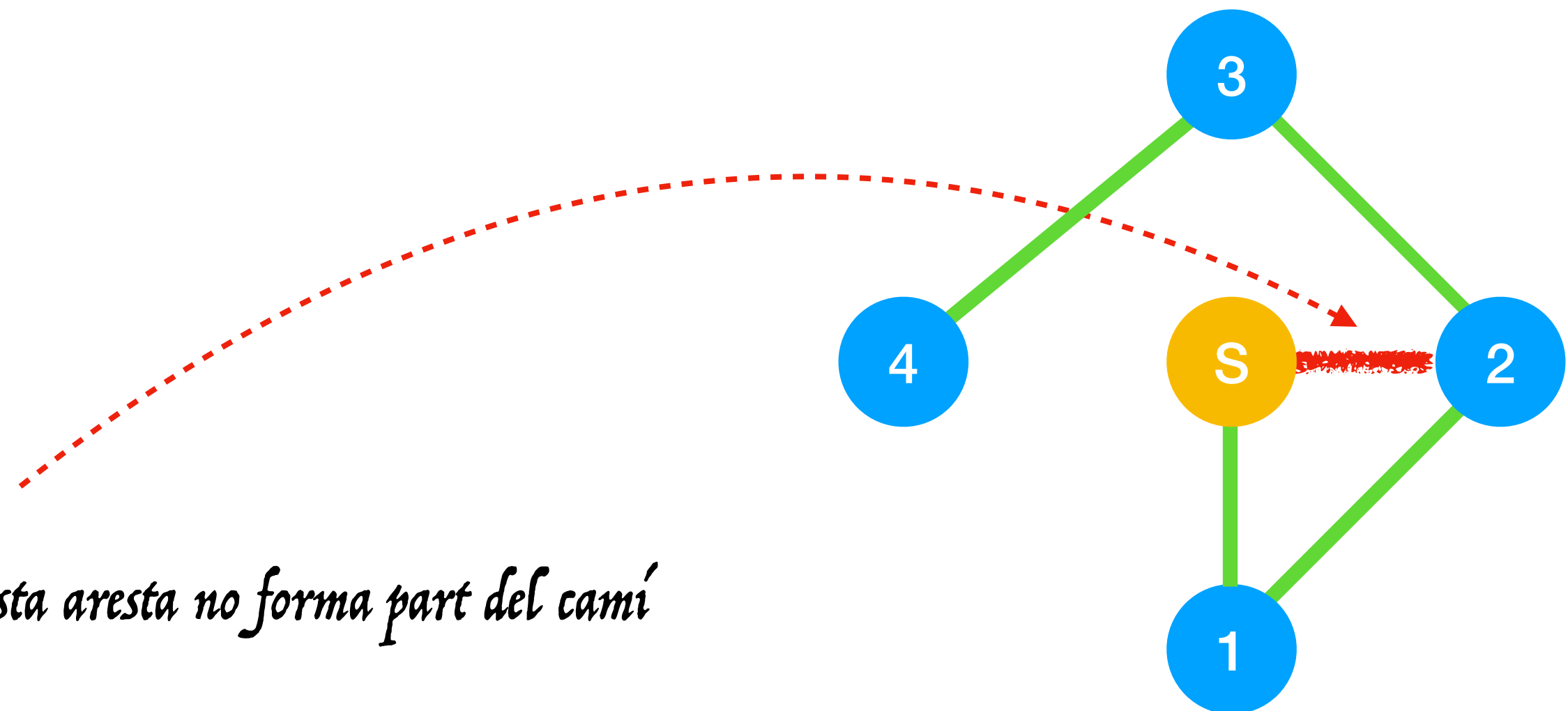
Camí Eulerià

- Un **camí Eulerià** és aquell camí que recorre **totes les arestes** d'un graf passant una i només una vegada per a cada una d'elles.
- No tots els grafs tenen un camí Eulerià
- El graf de la dreta té un camí Eulerià, però per trobar-lo hem de començar des d'un node en concret
 - Pot haver-hi més d'un camí



Camí Eulerià

- Un **camí Eulerià** és aquell camí que recorre **totes les arestes** d'un graf passant una i només una vegada per a cada una d'elles.
- No tots els grafs tenen un camí Eulerià
- El graf de la dreta té un camí Eulerià, però per trobar-lo hem de començar des d'un node en concret
 - Pot haver-hi més d'un camí

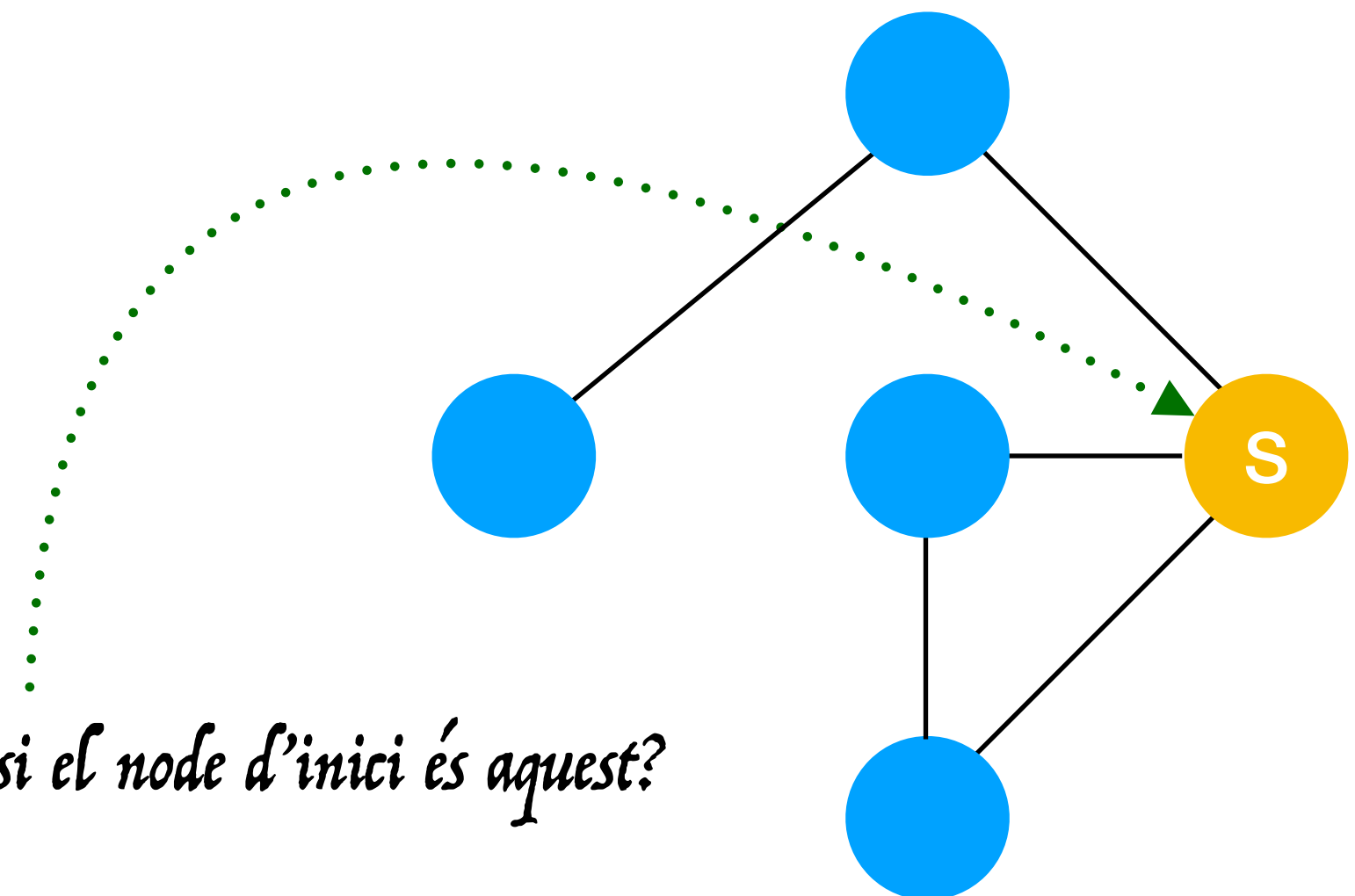


Això no és una sol·lució! Aquesta aresta no forma part del camí

Camí Eulerià

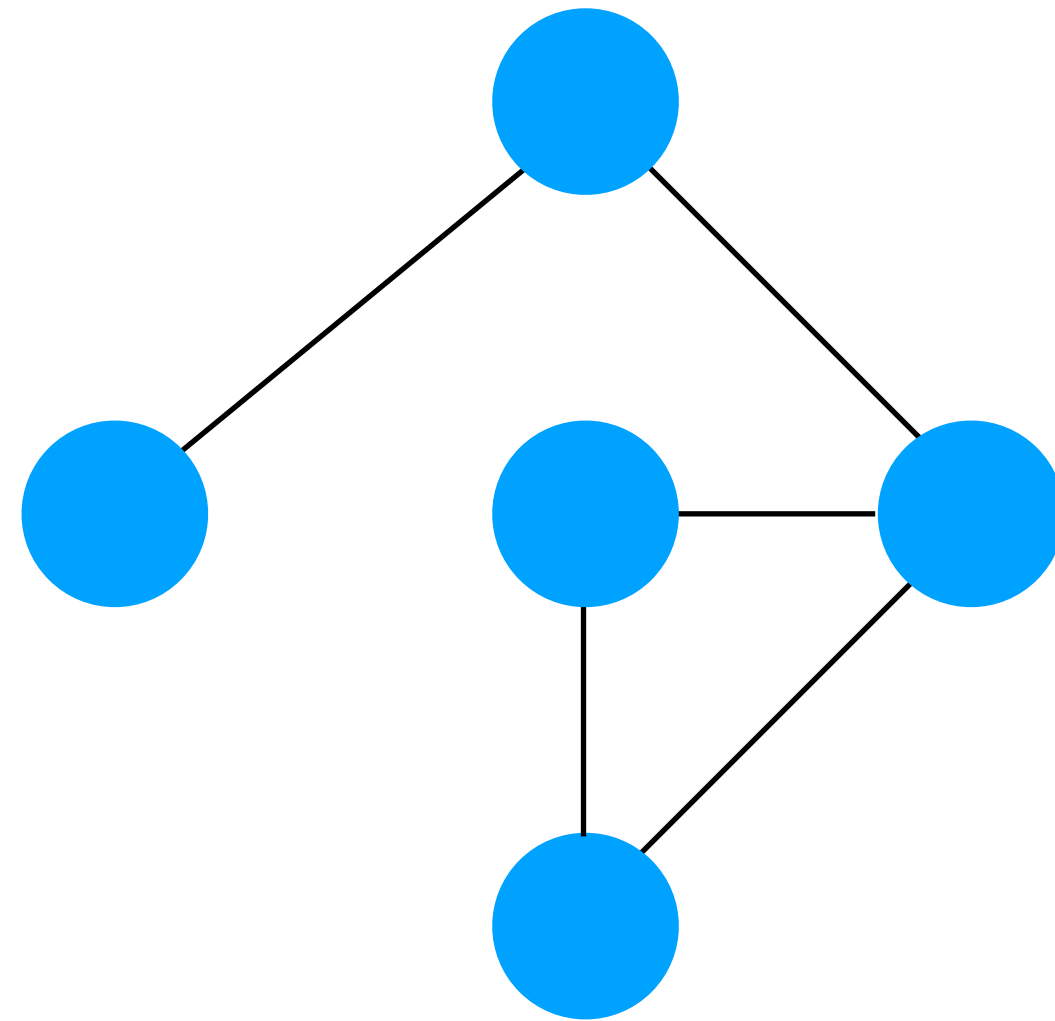
- Un **camí Eulerià** és aquell camí que recorre **totes les arestes** d'un graf passant una i només una vegada per a cada una d'elles.
- No tots els grafs tenen un camí Eulerià
- El graf de la dreta té un camí Eulerià, però per trobar-lo hem de començar des d'un node en concret
 - Pot haver-hi més d'un camí

Trobarem una sol·lució si el node d'inici és aquest?



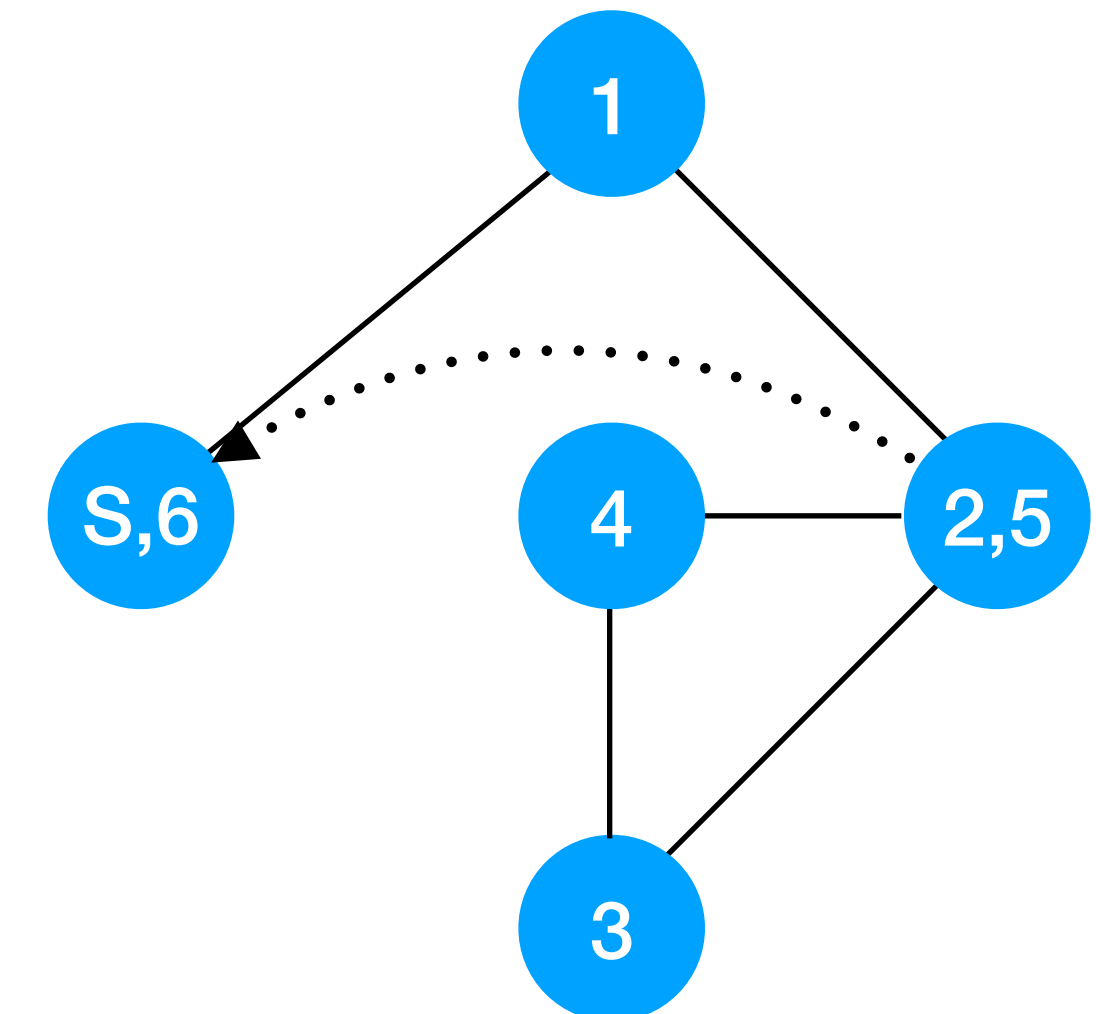
Camí/Circuit Eulerià

- Un **circuit (o cicle) Eulerià** és aquell **camí eulerià** que comença i acaba en el mateix **node**.



Aquest graf no té un circuit Eulerià!

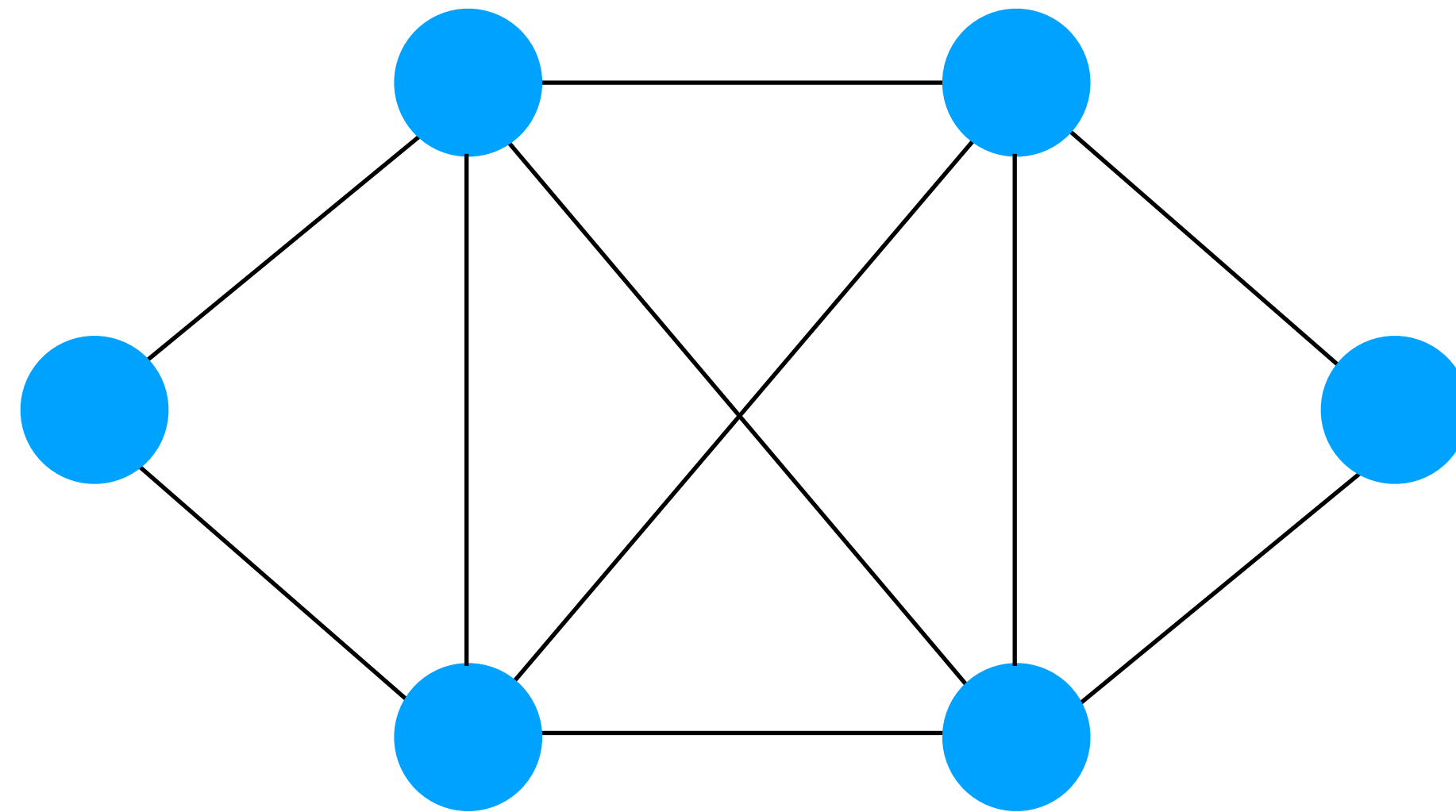
*com l'hauriem de modificar per tal que el graf
tingués un camí Eulerià?*



Aquí tenim una possibilitat!

Camí/Circuit Eulerià

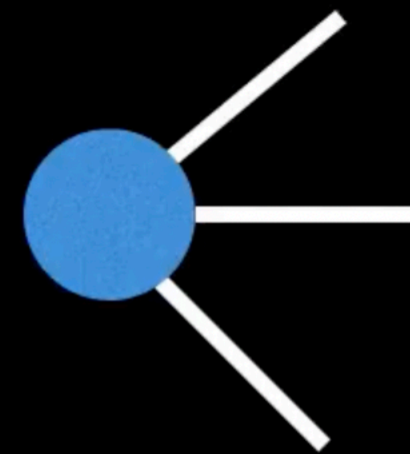
- Un **circuit (o cicle) Eulerià** és aquell **camí eulerià** que comença i acaba en el mateix **node**.



Té un circuit Eulerià aquest graf?

Què és el grau d'un node?

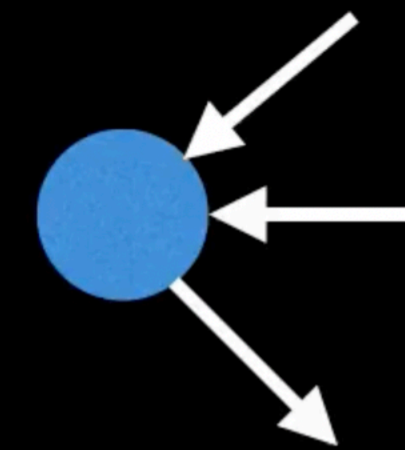
Graf no dirigit



Grau (degree) del node = 3

El grau d'un node és el nombre d'arestes incidents a ell

Graf dirigit



Grau d'entrada (In degree) = 2

Grau de sortida (Out degree) = 1

El grau d'entrada (indegree) es el nombre d'arestes d'entrada i el grau de sortida (outdegree) és el nombre d'arestes de sortida

Quines són les condicions necessàries per què el graf tingui un Camí/Circuit Eulerià?

Això depèn del tipus de graf que tinguem.

Camí Eulerià

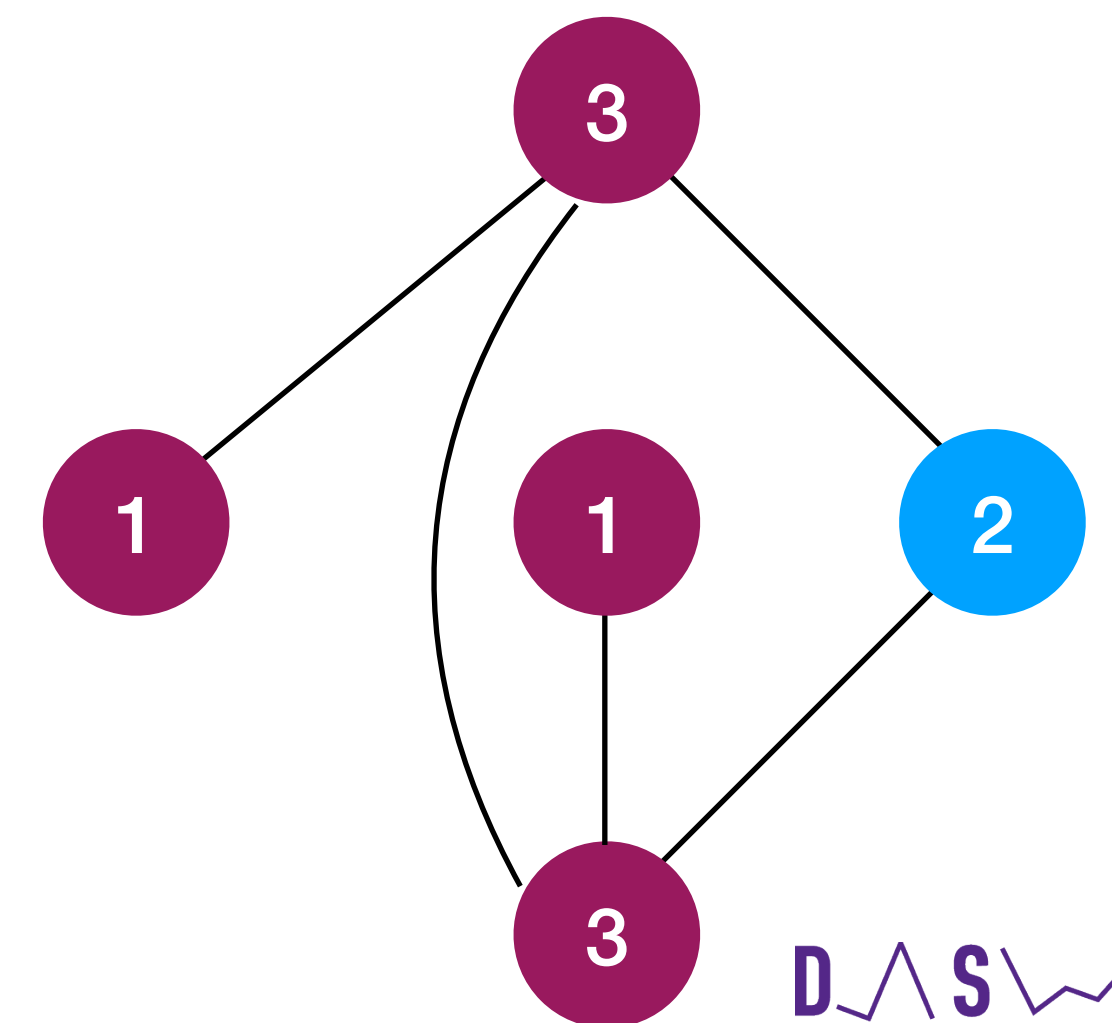
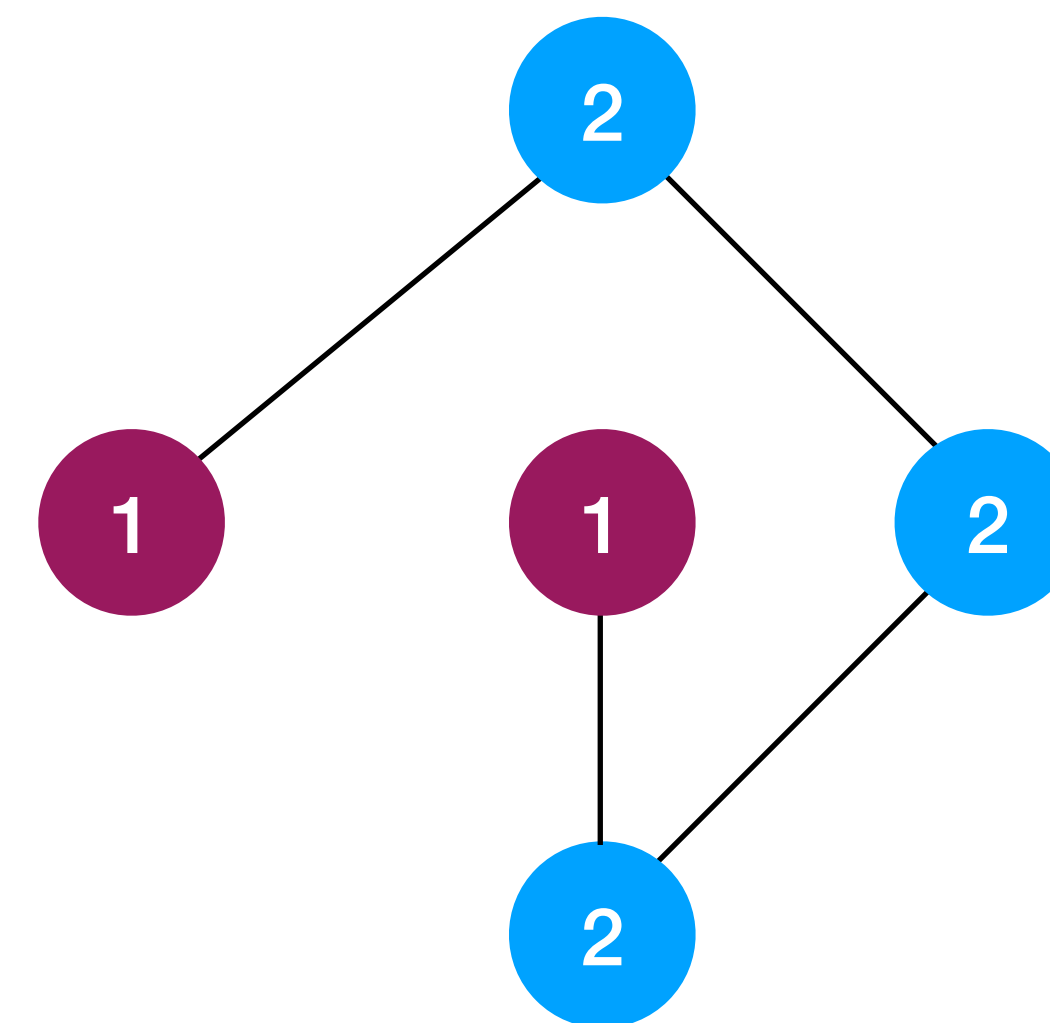
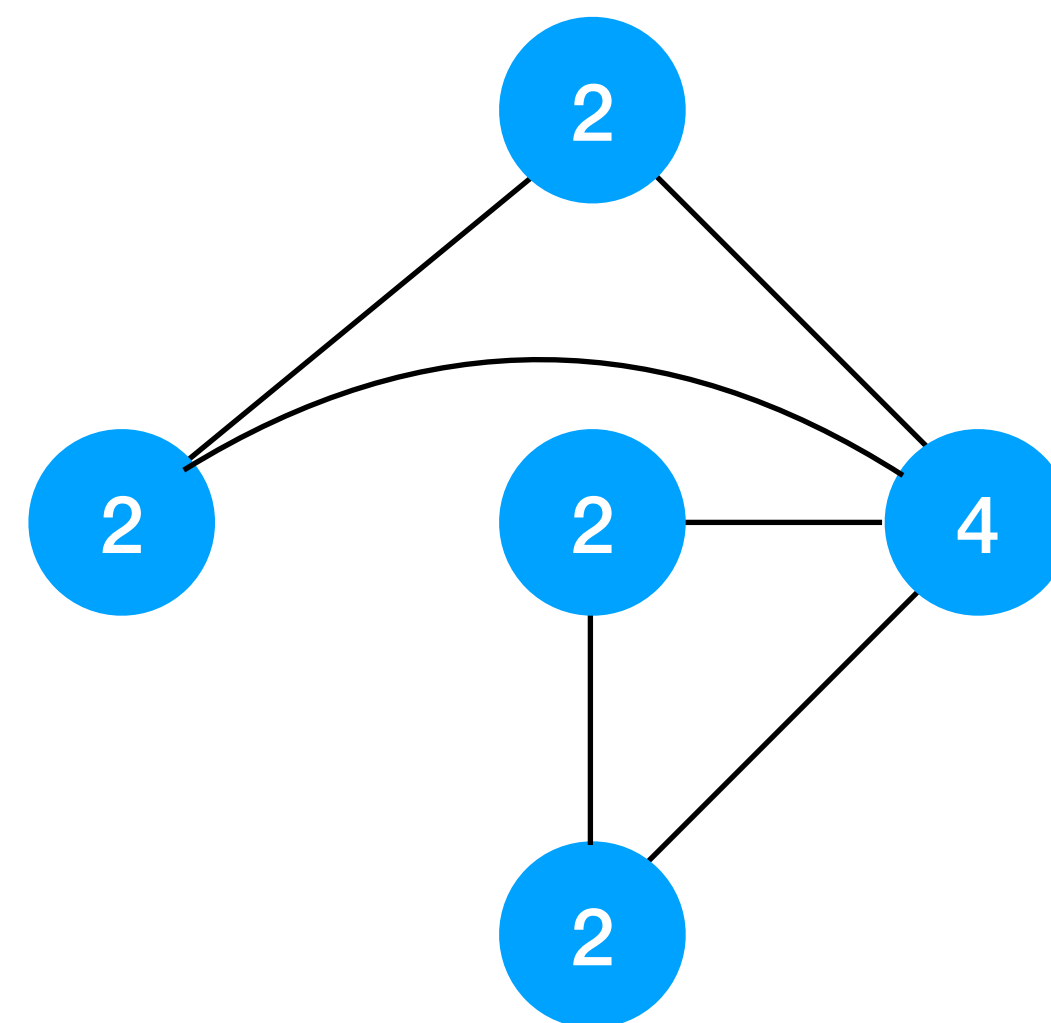
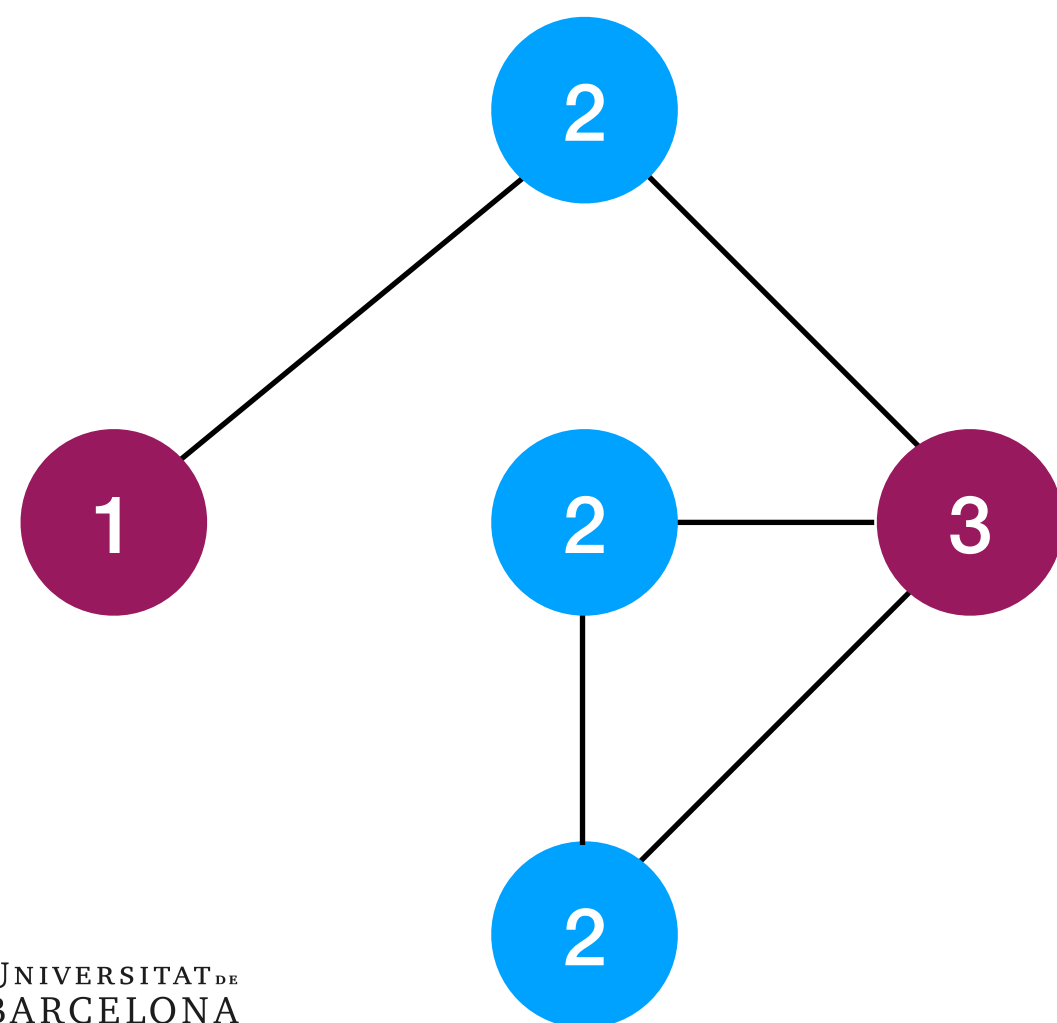
Circuit Eulerià

Graf no dirigit

Dues possibilitats:

- 1) Tots els **vèrtex** del graf tenen un **grau parell**
- 2) Hi ha **exactament 2** **vèrtex** amb **grau senar**

Tots els **vèrtex** del graf tenen un **grau parell**



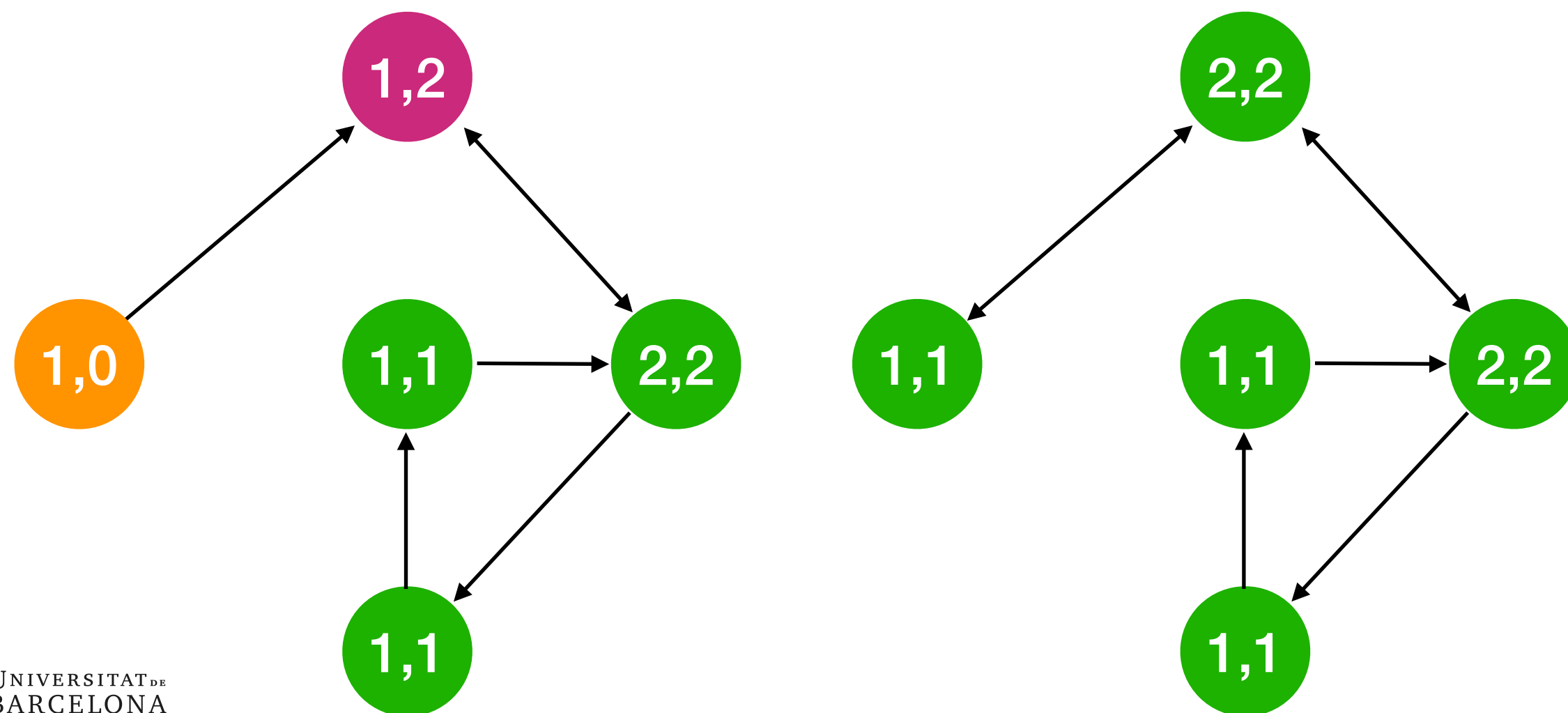
Quines són les condicions necessàries per què el graf tingui un Camí/Circuit Eulerià?

Això depèn del tipus de graf que tinguem.

Camí Eulerià

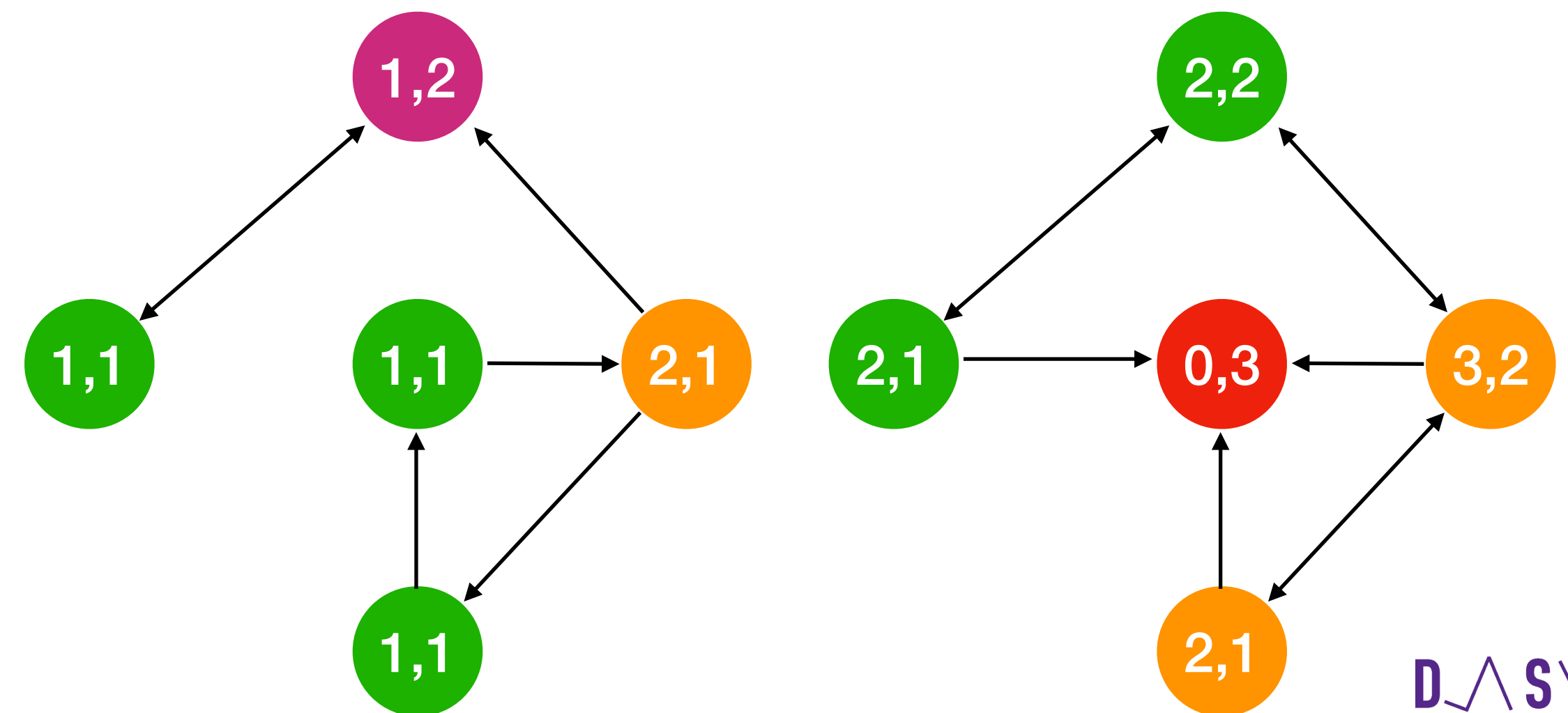
Graf dirigit

Màxim un vèrtex on $(\text{outdegree}) - (\text{indegree}) = 1$;
màxim un vèrtex $(\text{indegree}) - (\text{outdegree}) = 1$,
i tots els altres vèrtexs tenen el **mateix grau d'entrada i sortida**

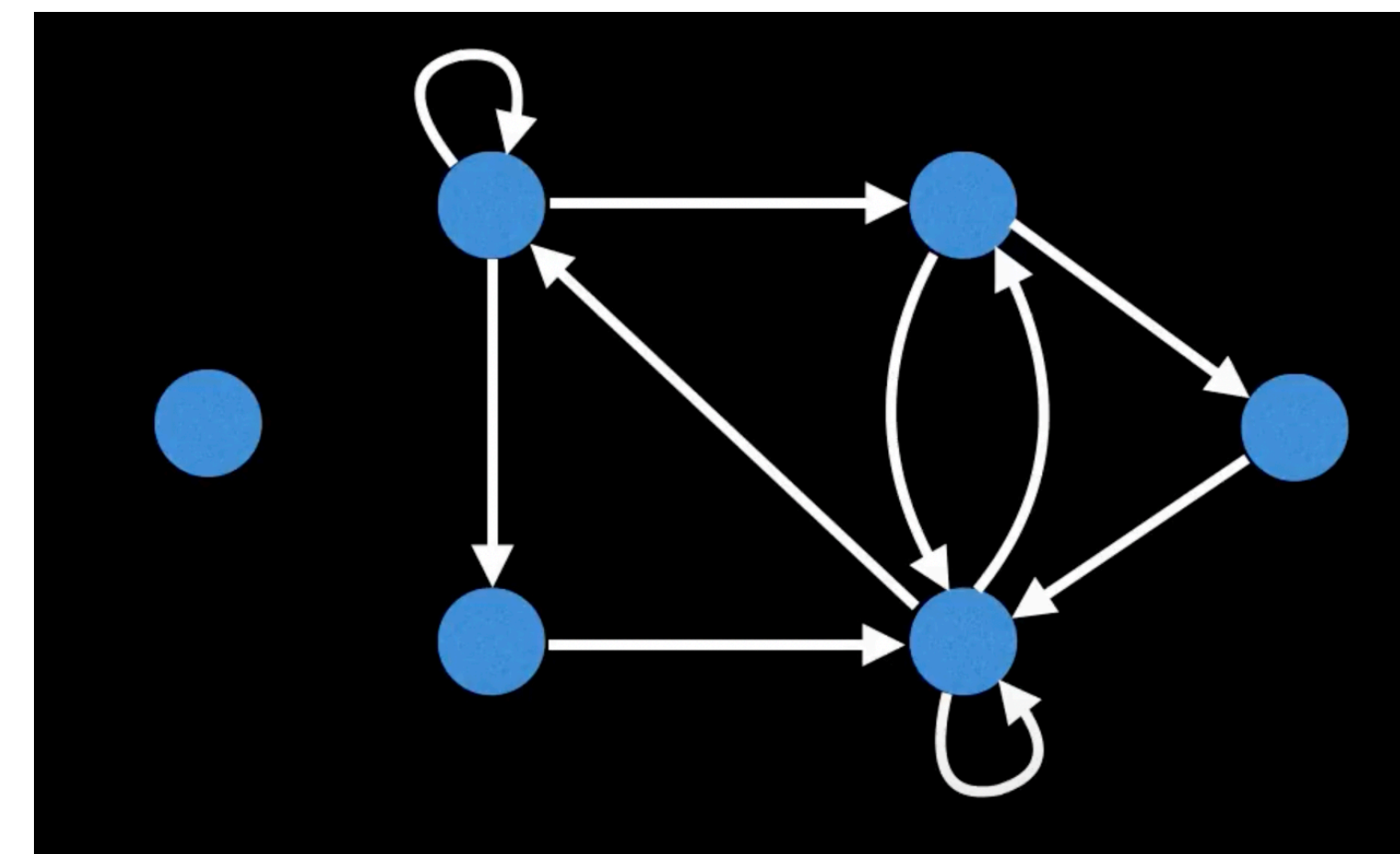
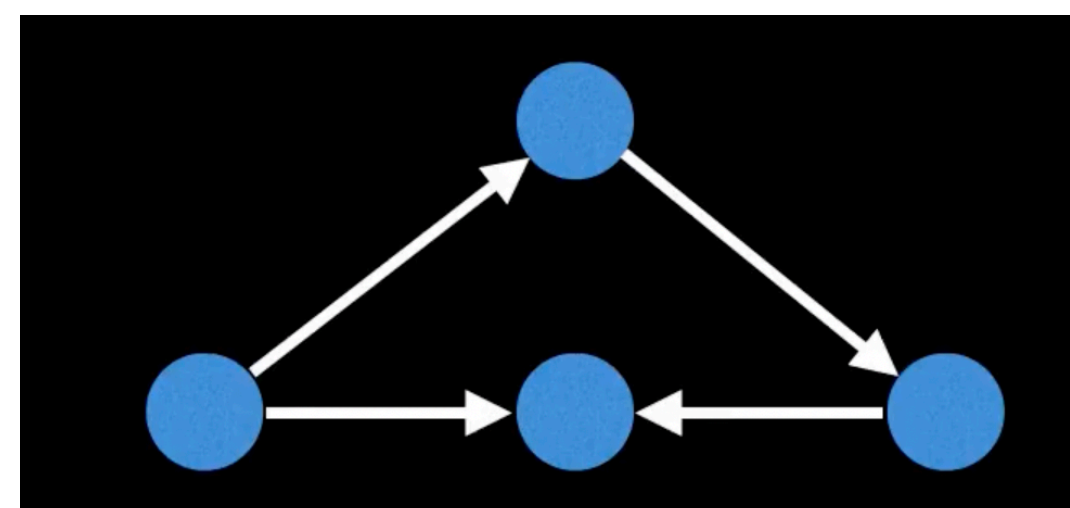
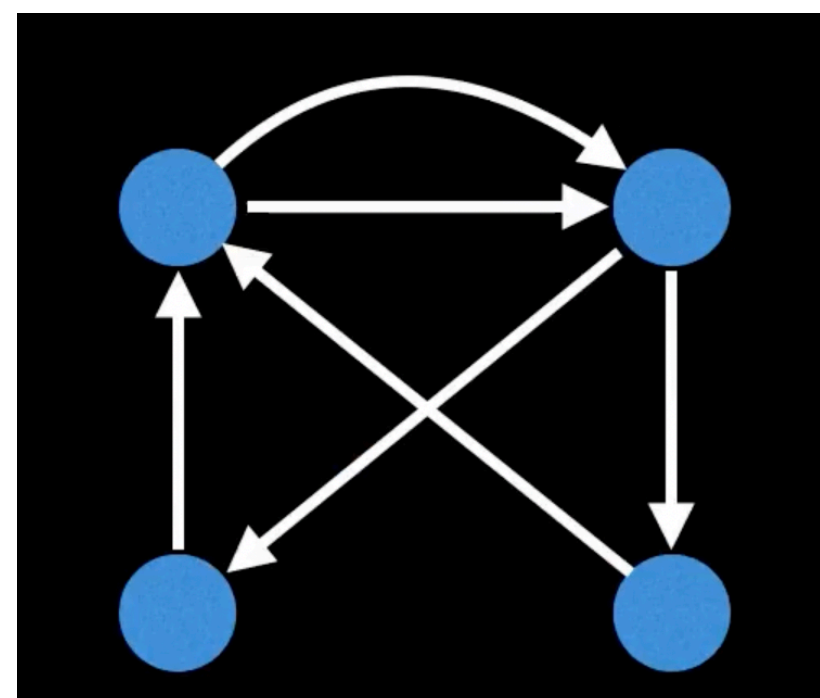
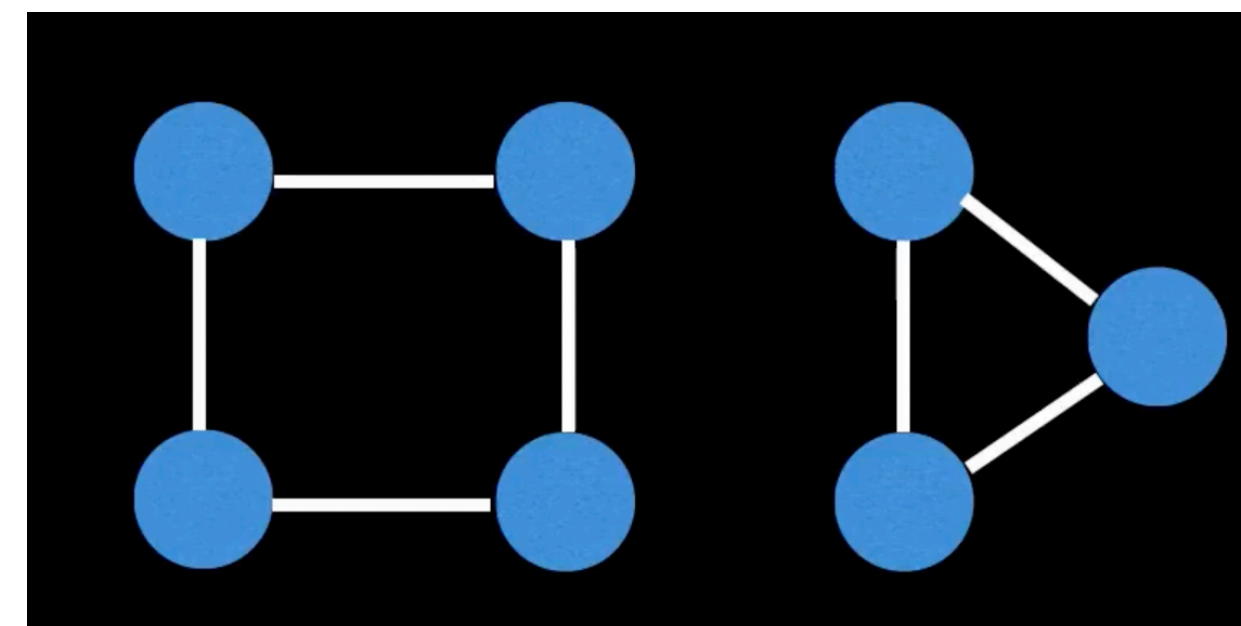
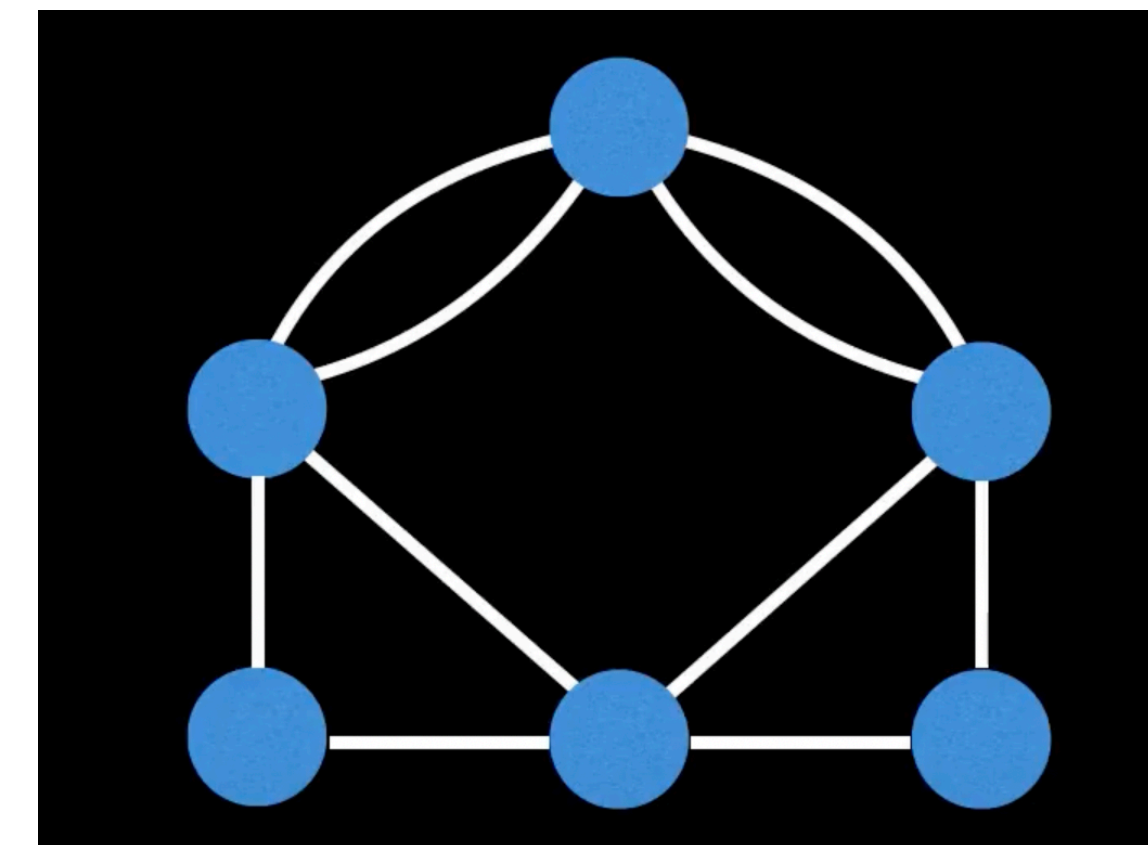
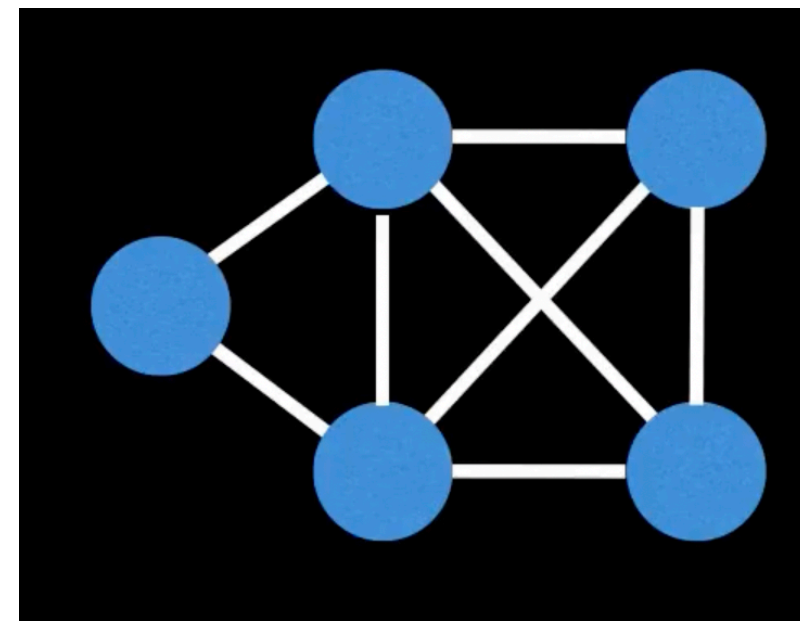
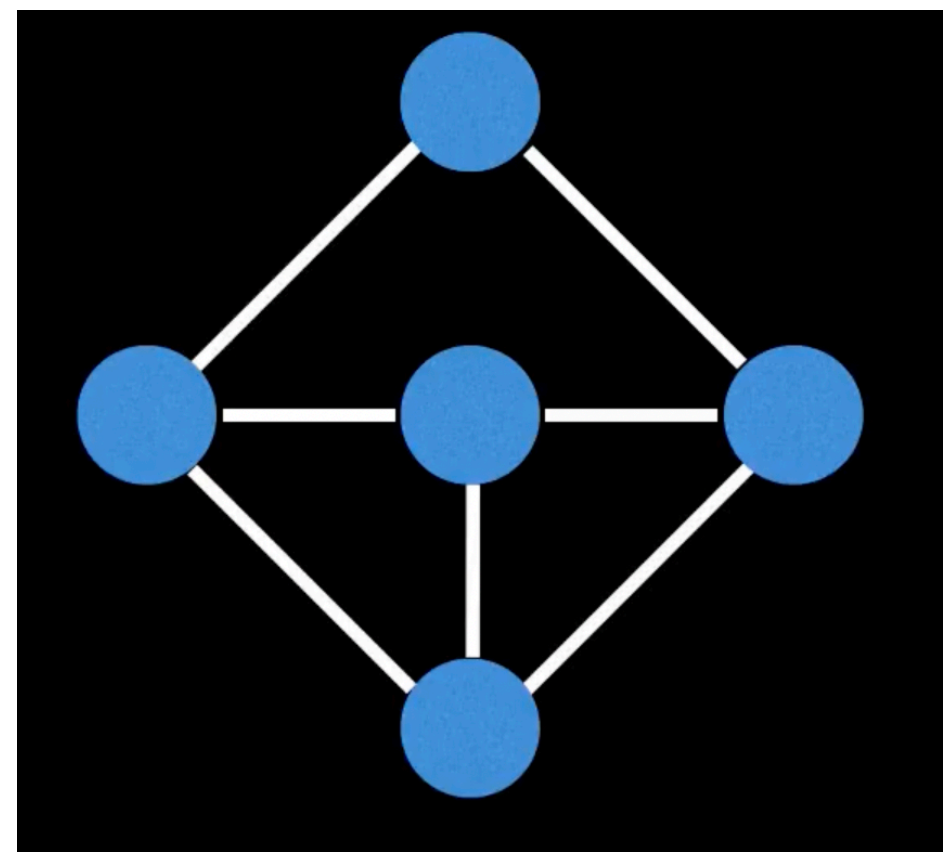


Circuit Eulerià

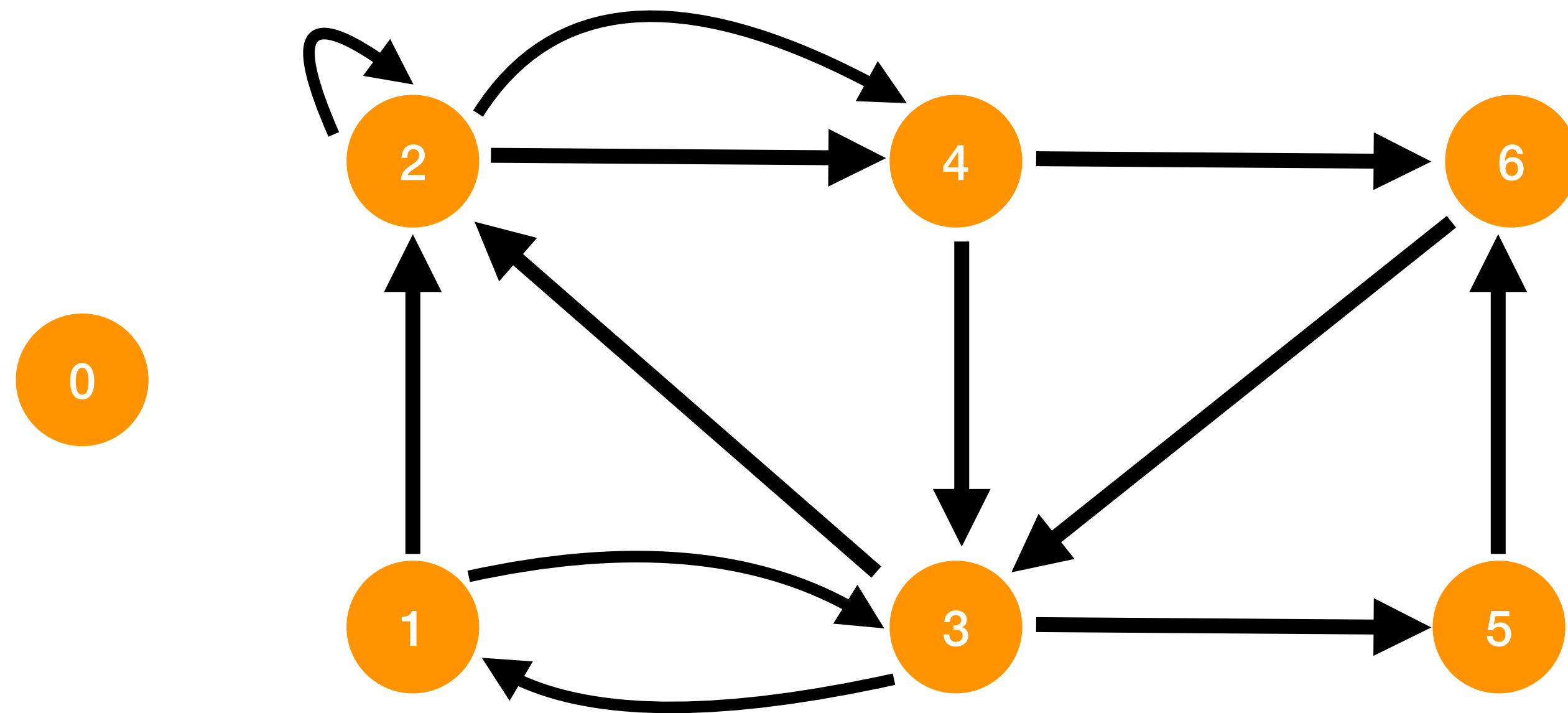
Tots els **vèrtex** del graf compleixen:
 $\text{indegree} == \text{outdegree}$



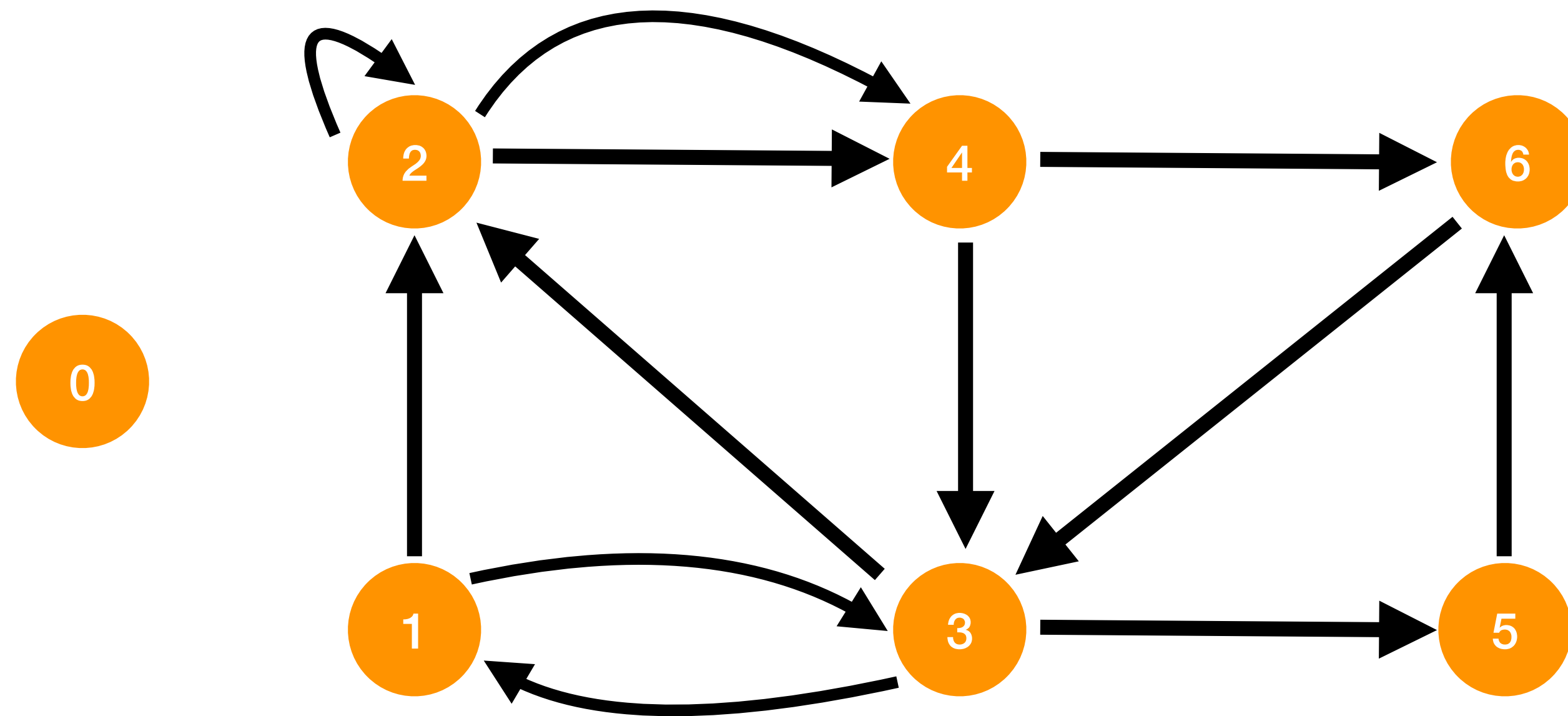
Quin dels següents grafs tenen un camí i/o circuit eulerià?



Exemple



Exemple - Trobar el Camí Eulerià



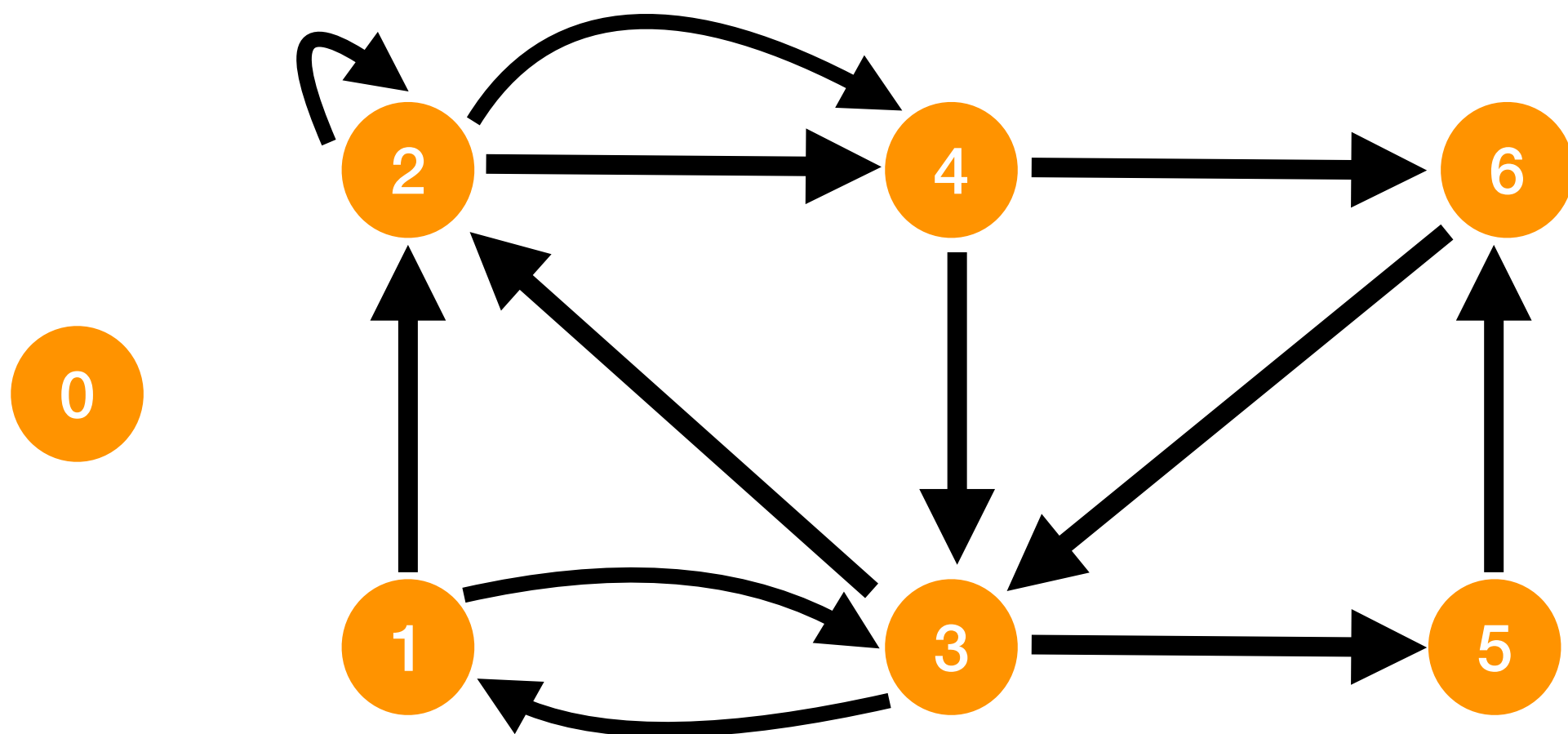
Pas 1: Determinar si existeix un camí eulerià.

Exemple - Trobar el Camí Eulerià

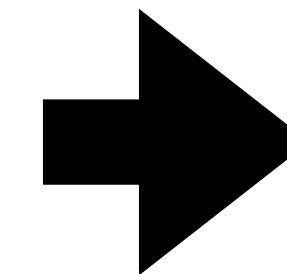
Pas 1: Determinar si existeix un camí eulerià.

Existeix si:

Màxim un vèrtex on **(outdegree) - (indegree) = 1**;
màxim un vèrtex **(indegree) - (outdegree) = 1**,
i tots els altres vèrtexs tenen el **mateix grau d'entrada i sortida**



Node	In	Out
0	0	0
1	1	2
2	3	3
3	3	3
4	2	2
5	1	1
6	2	1



Existeix!

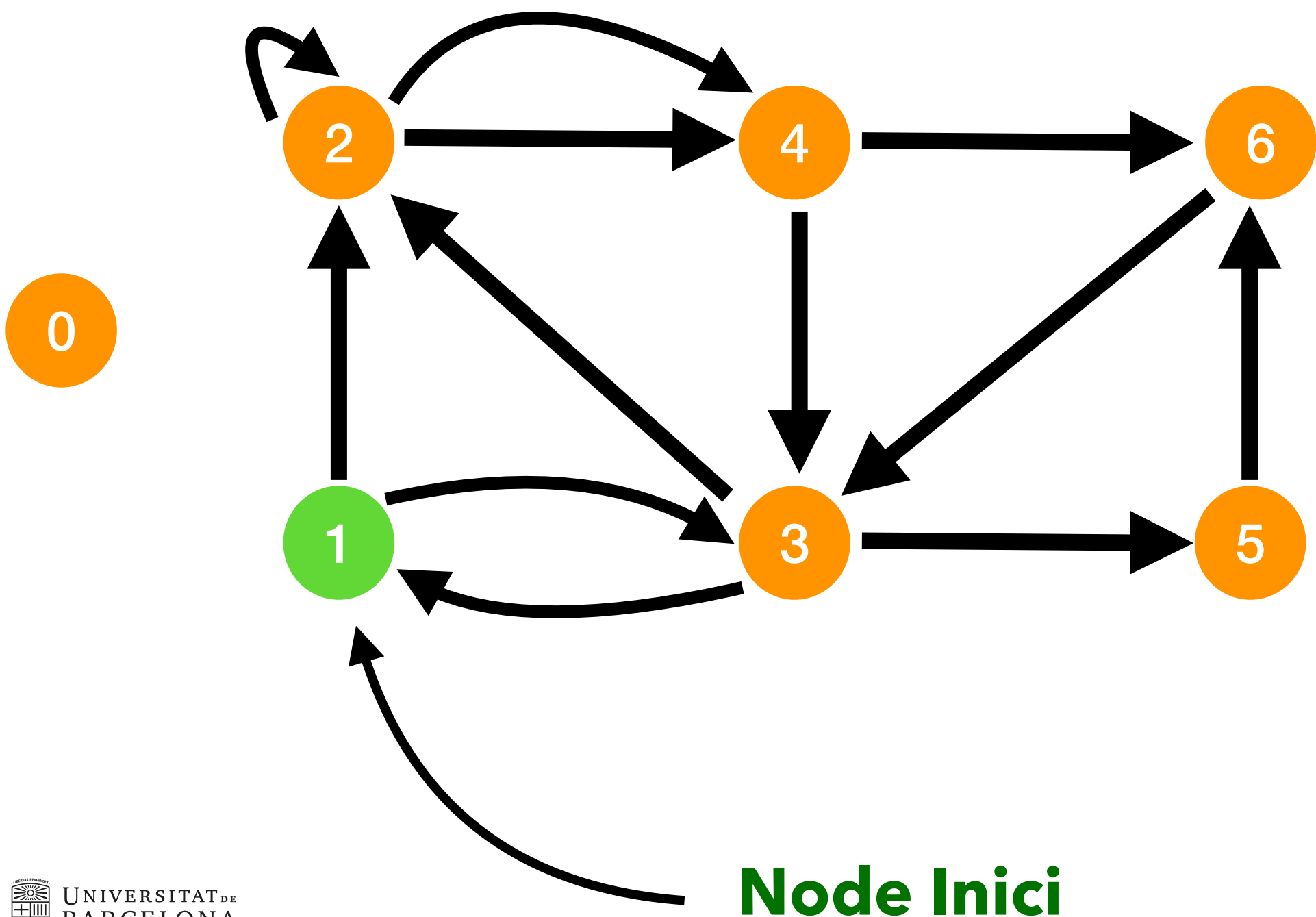
Exemple - Trobar el Camí Eulerià

Pas 2: Determinar node d'inici

Si hi ha 0 vèrtexs amb grau imparells, comencem en qualsevol lloc.

Grafs No dirigits: Si hi ha 2 vèrtexs amb grau senars, comencem per un d'ells.

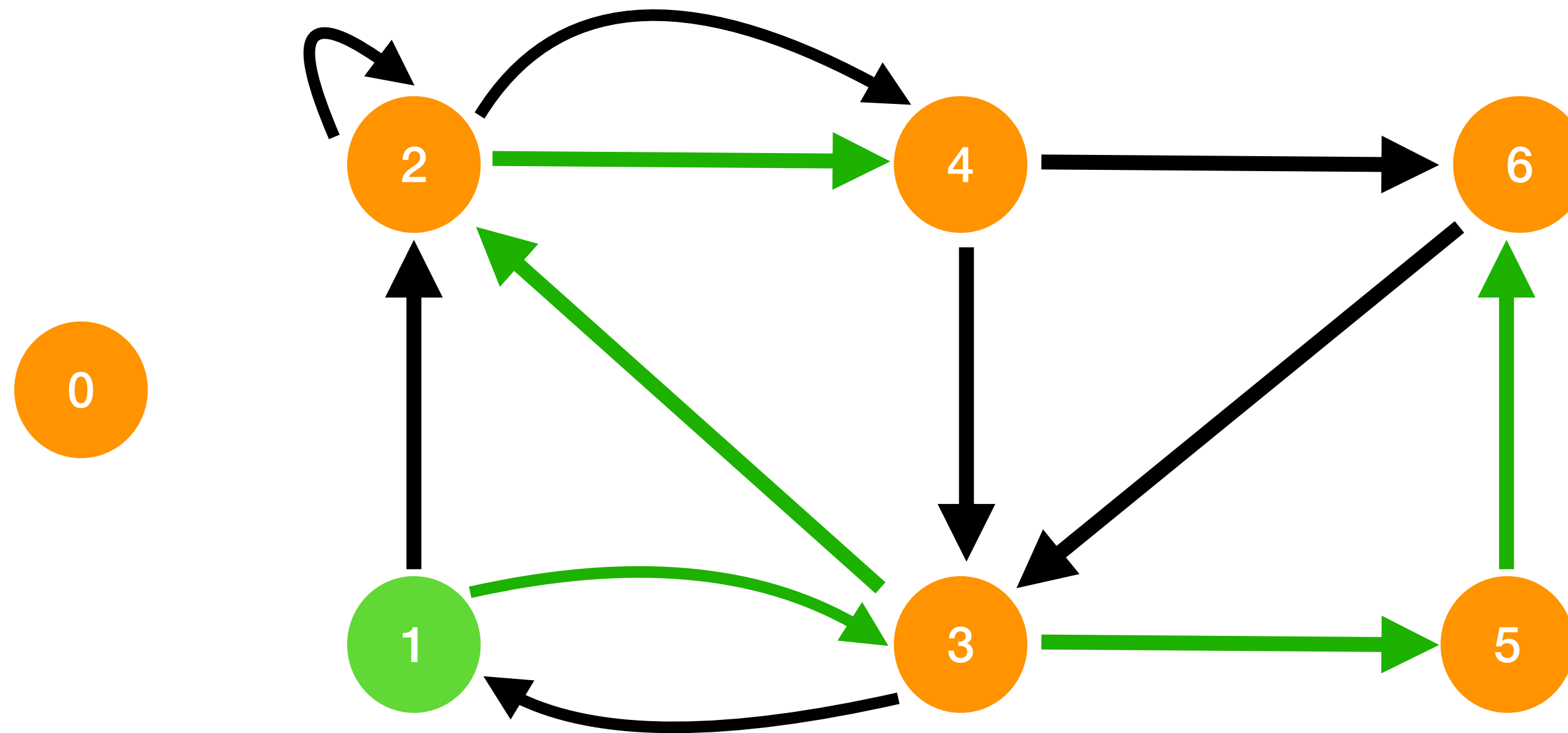
Graf Dirigit: Comencem pel node amb **(outdegree) - (indegree) = 1**



Node	In	Out
0	0	0
1	1	2
2	3	3
3	3	3
4	2	2
5	1	1
6	2	1

Exemple - Trobar el Camí Eulerià

Pas 3: Trobar el camí Eulerià



Que passa si apliquem el DFS?

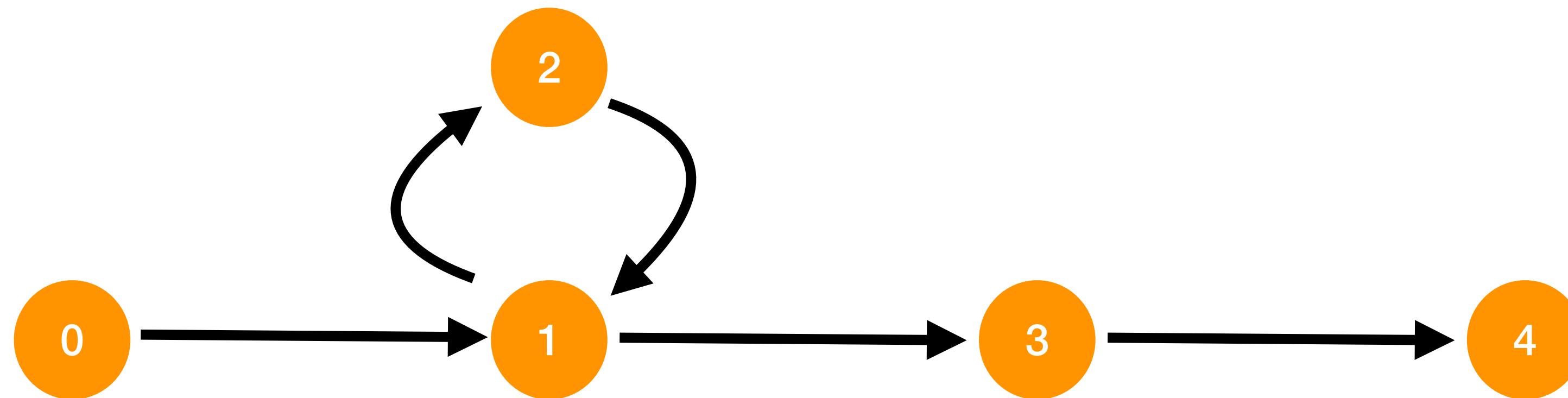
seleccionant aleatòriament les arestes durant el DFS, trobarem un camí que va del node inicial fins al node final

No trobem la solució ja que hi ha arestes que no son visitades

Exemple - Trobar el Camí Eulerià

- Hem de modificar el DFS per tal que forcem a visitar totes les Arestes

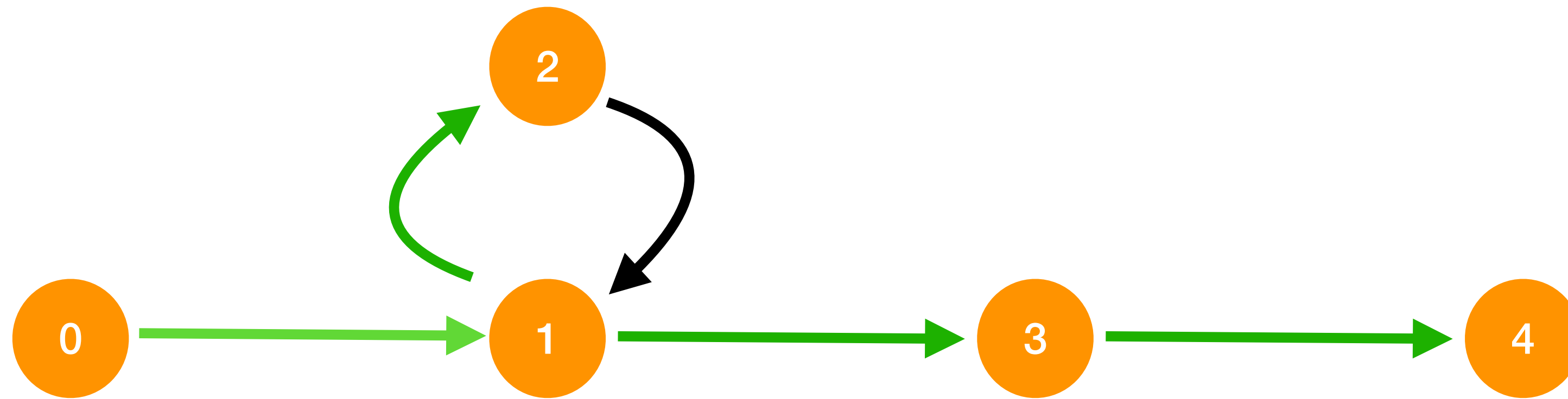
Per il·lustrar el problema, considerem el següent graf on el node d'inici es el node 0



Exemple - Trobar el Camí Eulerià

- Hem de modificar el DFS per tal que forcem a visitar totes les Arestes

Per il·lustrar el problema, considerem el següent graf on el node d'inici es el node 0

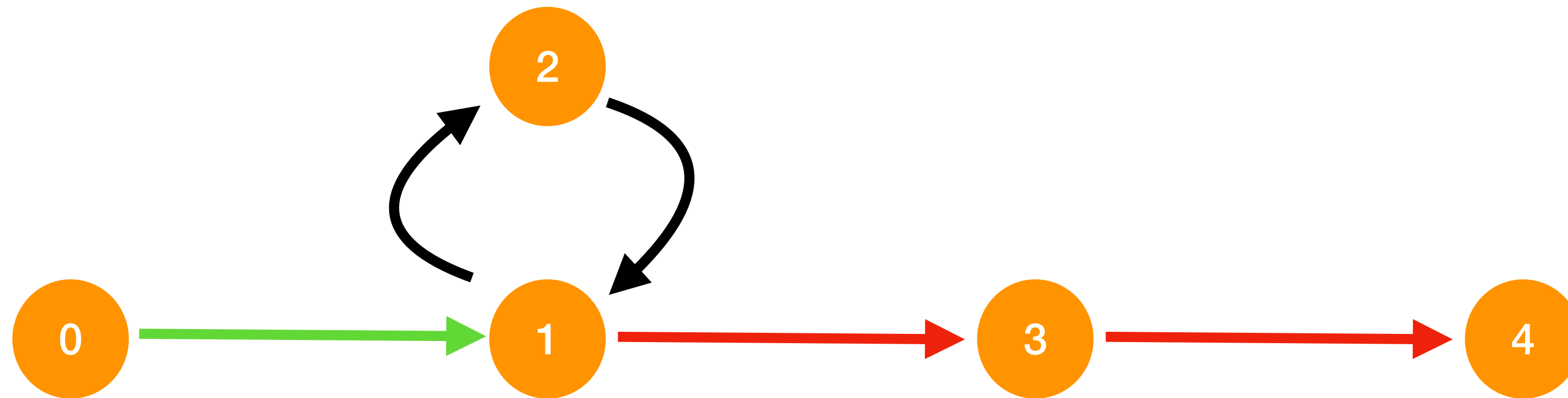


El DFS es podria donar el següent resultat: **0, 1, 3, 4, 2**

Exemple - Trobar el Camí Eulerià

- Hem de modificar el DFS per tal que forcem a visitar totes les Arestes

Per il·lustrar el problema, considerem el següent graf on el node d'inici es el node 0

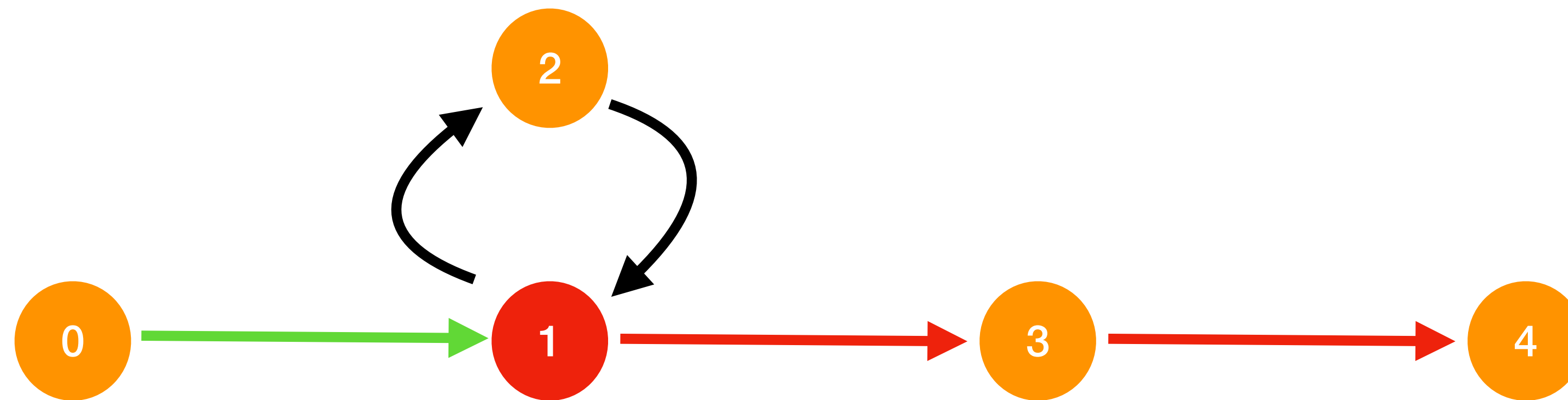


Modifiquem el DFS tal que quan fem el backtracking (no queden nodes adjacents sense visitar) afegim el node dins la solució. **El resultat seria:** [4,3]

Exemple - Trobar el Camí Eulerià

- Hem de modificar el DFS per tal que forcem a visitar totes les Arestes

Per il·lustrar el problema, considerem el següent graf on el node d'inici es el node 0

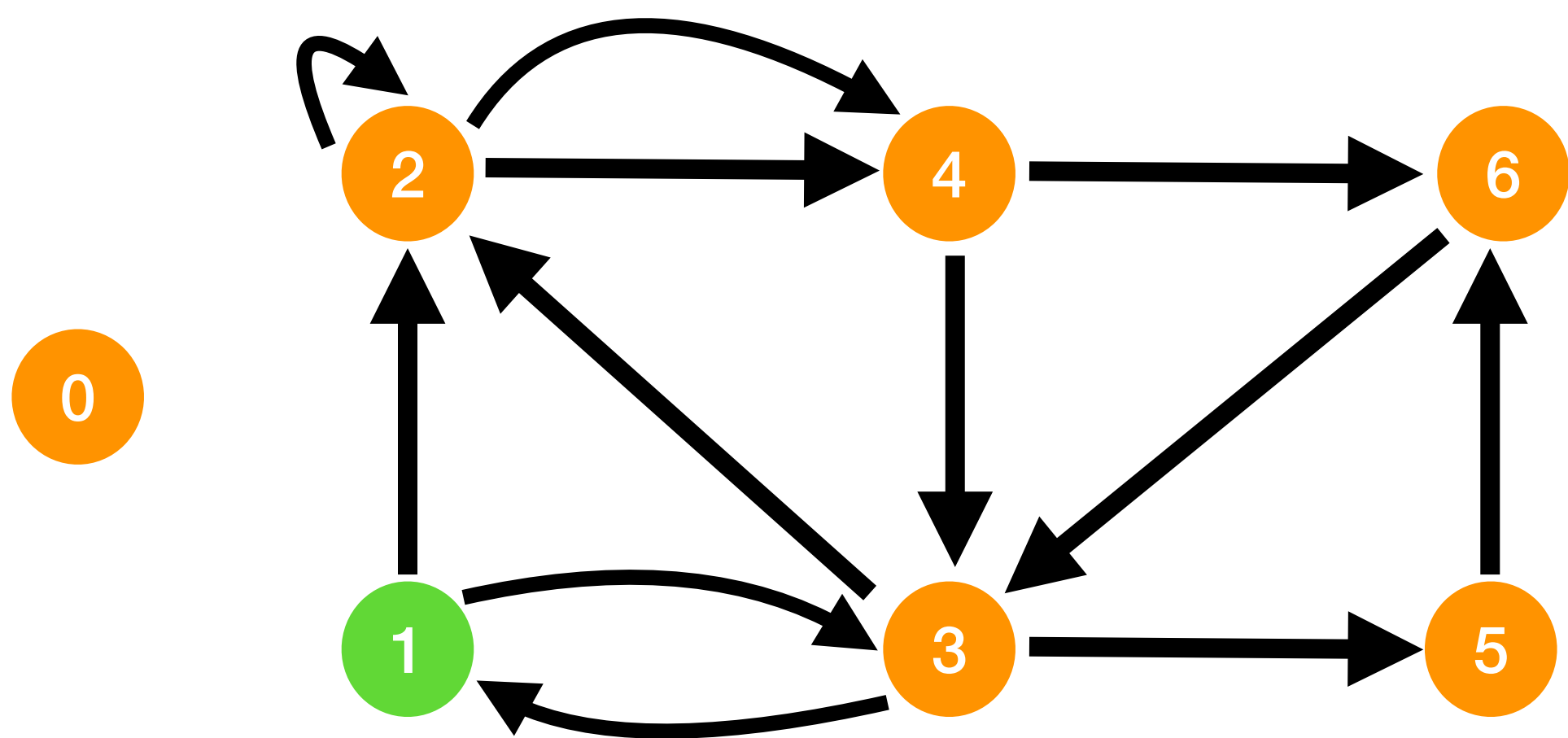


Modifiquem el DFS tal que quan fem el backtracking (no queden nodes adjacents sense visitar) afegim el node dins la solució. **El resultat seria:** [4,3]

Si al fer backtracking, ens trobem amb un node que té fills sense explorar, els explorem utilitzant el DFS i afegim els nodes a la solució com hem vist abans

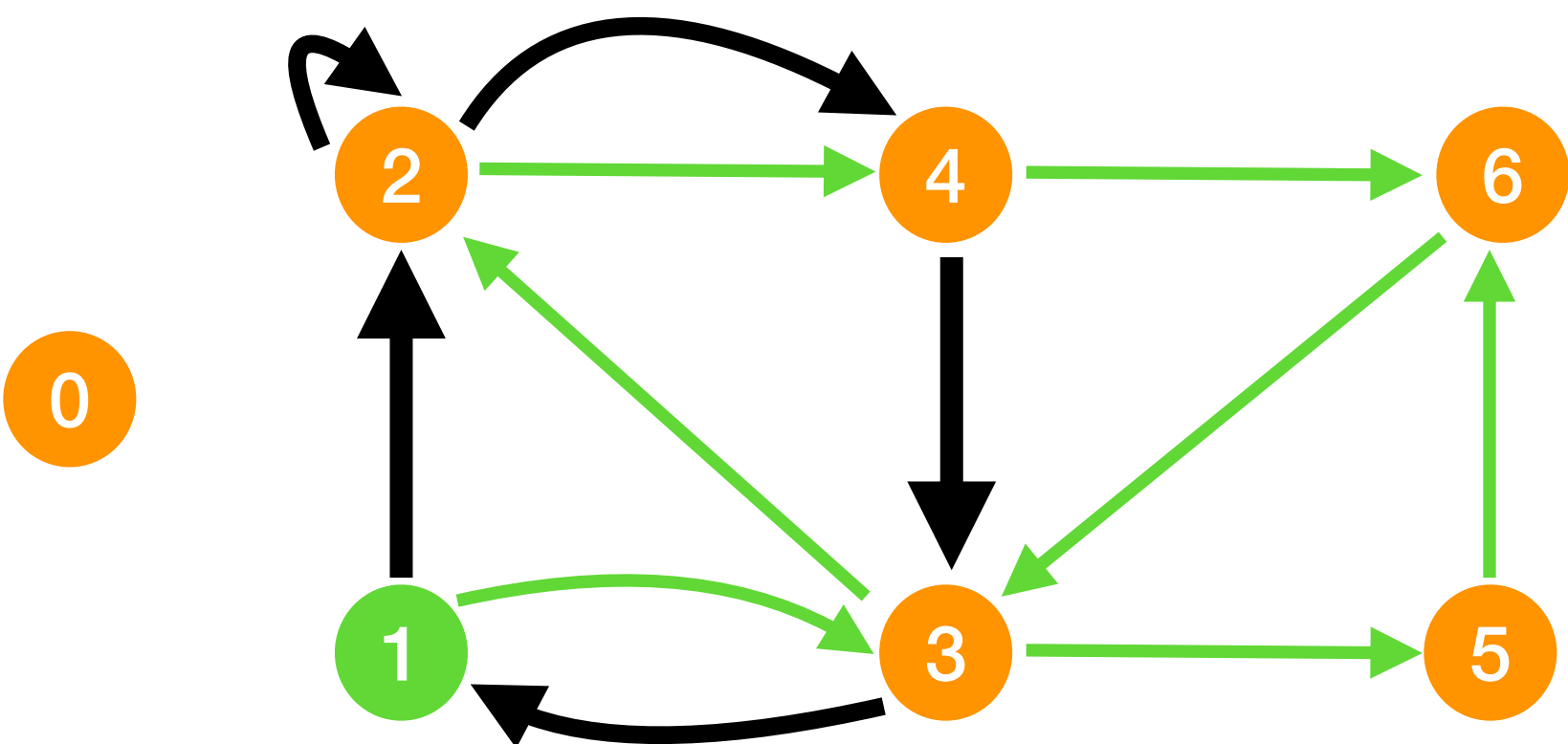
El resultat seria: [4,3,1,2,1,0]

Exemple - Trobar el Camí Eulerià



Node	In	Out
0	0	0
1	1	2
2	3	3
3	3	3
4	2	2
5	1	1
6	2	1

Executem DFS i fem la següent exploració:
1, 3, 5, 6, 3, 2, 4, 6



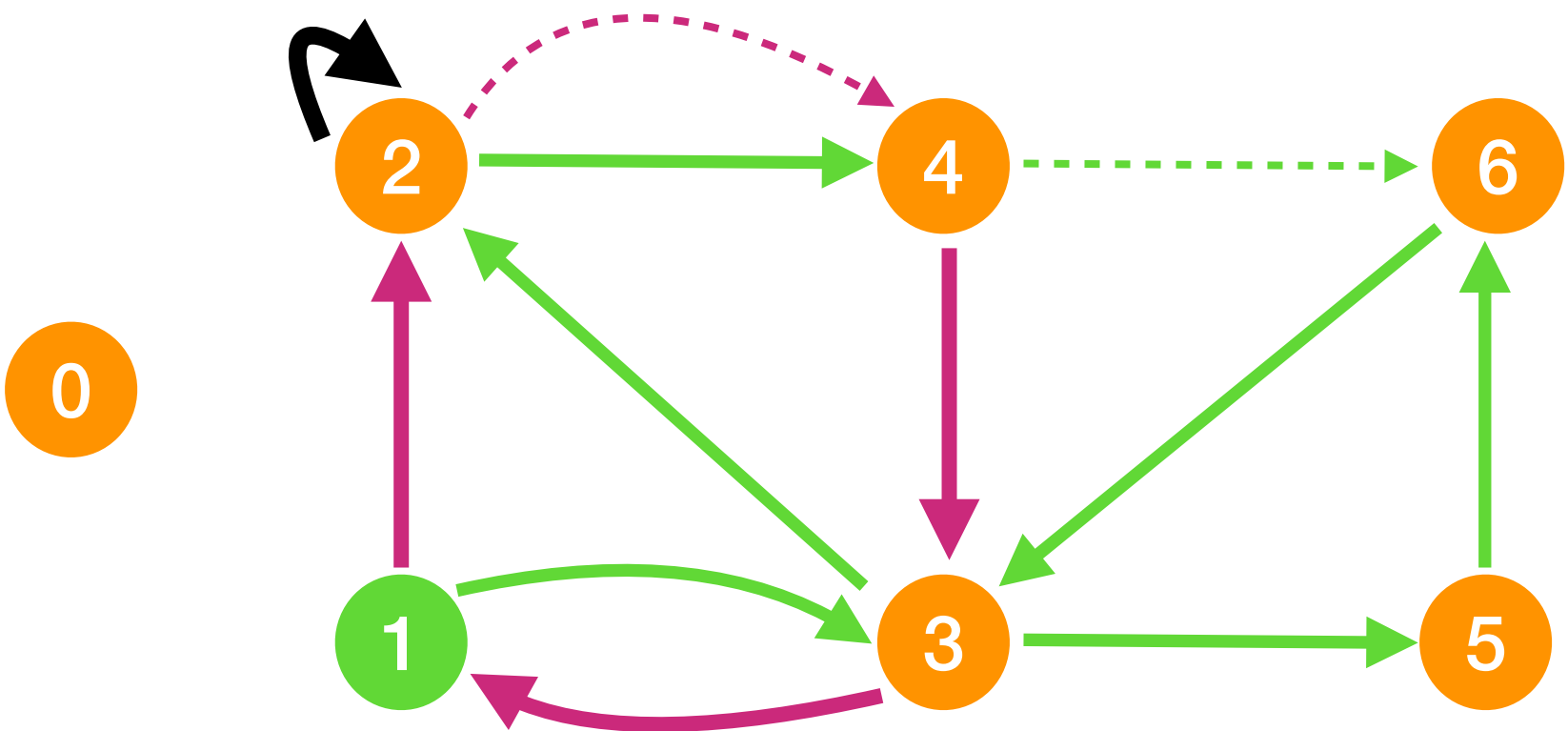
Node	Out
0	0
1	1
2	2
3	1
4	1
5	0
6	0

En lloc de mirar si el node està visitat
mirem si l'outdegree del node és major
que 0.
Al mateix temps que visitem els nodes,
reduïm el seu out-degree

Exemple - Trobar el Camí Eulerià

Executem DFS i fem la següent exploració:

1, 3, 5, 6, 3, 2, 4, 6



Quan el DFS no té més nodes per explorar (es, a dir, ens trobem un node on $out[i]=0$), fem backtracking i afegim el node a la solució

Solució: [6]

Tornem al node 4, i veiem que aquest node té arestes sense visitar ($out[i]>0$). Cridem el DFS a partir del node 4

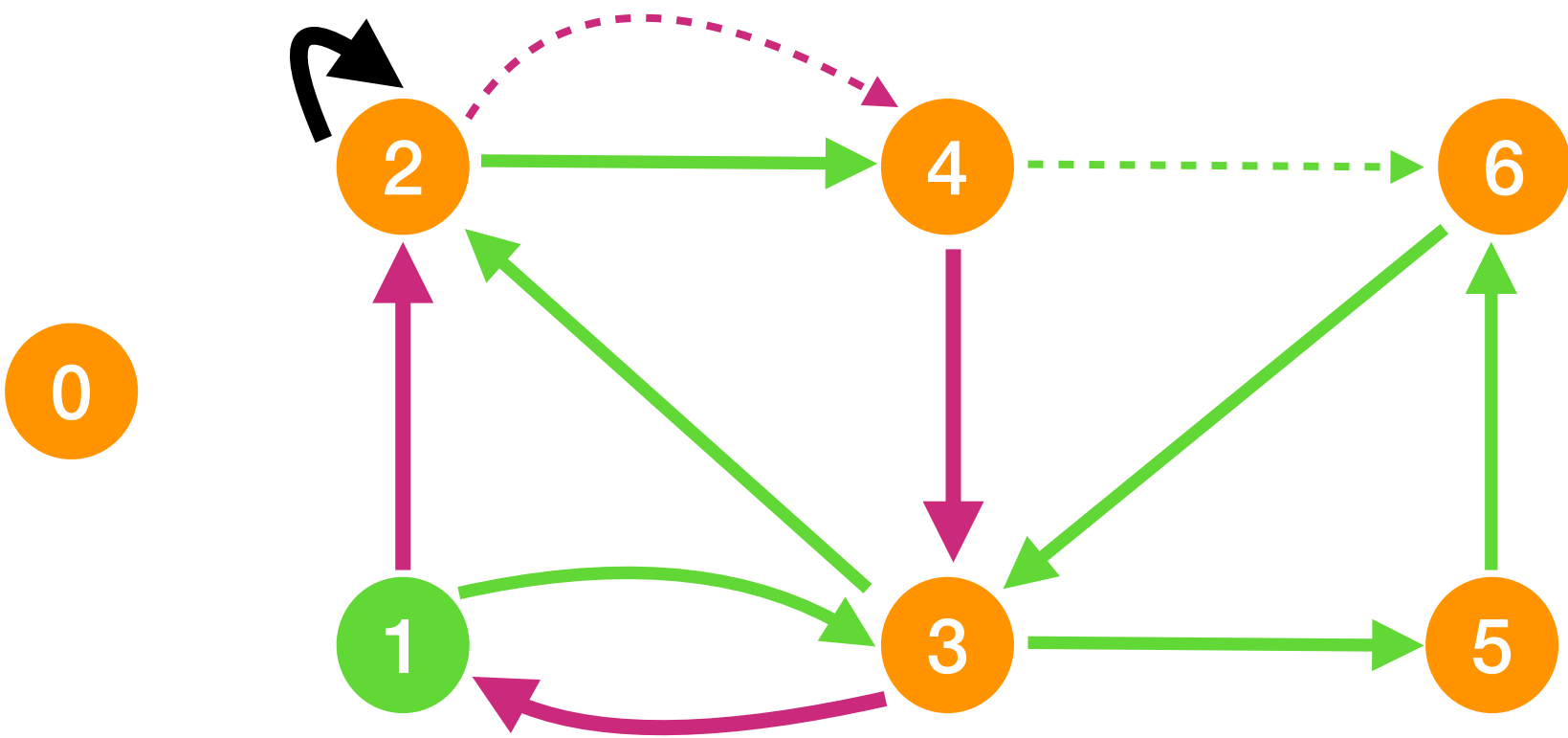
Executem DFS (4) i fem la següent exploració:

4, 3, 1, 2, 4

Quan el DFS no té més nodes per explorar (es, a dir, ens trobem un node on $out[i]=0$), fem backtracking i afegim el node a la solució

Solució: [4, 6]

Exemple - Trobar el Camí Eulerià



Node	Out
0	0
1	1
2	2
3	1
4	1
5	1
6	0

Tornem al node 2, i veiem que aquest node té arestes sense visitar ($out[i]>0$). Cridem el DFS a partir del node 2

Executem DFS (2).

No tenim nodes sense visitar, i per tant fem backtraking. Al fer backtracking afegim el node a la sol·lució

Solució: [2, 4, 6]

Continuem el backtracking de la crida del **DFS(4)**

Solució: [4, 3, 1, 2, 2, 4, 6]

Continuem el backtracking de la crida del **DFS(1)**

Solució: [1, 3, 5, 6, 3, 2, 4, 3, 1, 2, 2, 4, 6]

Com trobar el Camí Eulerià

- Pas 1:
 - Determinar si existeix un camí eulerià.
- Pas 2:
 - Busquem un node d'inici vàlid.
 - Si hi ha 0 vèrtexs amb grau imparell, comencem en qualsevol lloc.
 - Si hi ha 2 vèrtexs amb grau senar, comenceu per un d'ells.
- Pas 3
 - Executem la versió modificada del DFS

```

# Global/class scope variables
n = number of vertices in the graph
m = number of edges in the graph
g = adjacency list representing directed graph

in  = [0, 0, ..., 0, 0] # Length n
out = [0, 0, ..., 0, 0] # Length n

path = empty integer linked list data structure

function findEulerianPath():

    countInOutDegrees()
    if not graphHasEulerianPath(): return null

    dfs(findStartNode())

    # Return eulerian path if we traversed all the
    # edges. The graph might be disconnected, in which
    # case it's impossible to have an euler path.
    if path.size() == m+1: return path
    return null

```

```
function countInOutDegrees():  
    for edges in g:  
        for edge in edges:  
            out[edge.from]++  
            in[edge.to]++  
  
function graphHasEulerianPath():  
    start_nodes, end_nodes = 0, 0  
    for (i = 0; i < n; i++):  
        if (out[i] - in[i]) > 1 or (in[i] - out[i]) > 1:  
            return false  
        else if out[i] - in[i] == 1:  
            start_nodes++  
        else if in[i] - out[i] == 1:  
            end_nodes++  
    return (end_nodes == 0 and start_nodes == 0) or  
           (end_nodes == 1 and start_nodes == 1)
```

```
function findStartNode():
    start = 0
    for (i = 0; i < n; i = i + 1):
        # Unique starting node
        if out[i] - in[i] == 1: return i

        # Start at any node with an outgoing edge
        if out[i] > 0: start = i
    return start

function dfs(at):
    # While the current node still has outgoing edges
    while (out[at] != 0):

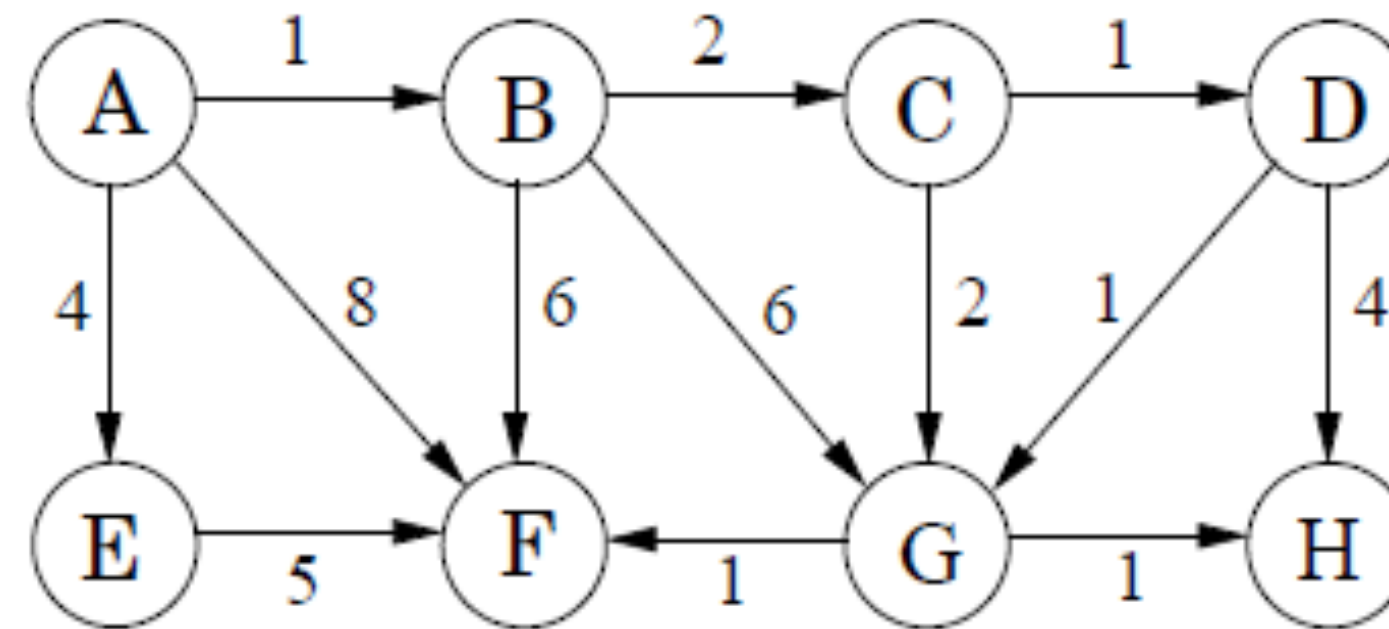
        # Select the next unvisited outgoing edge
        next_edge = g[at].get(--out[at])
        dfs(next_edge.to)

    # Add current node to solution
    path.insertFirst(at)
```

Exercici: Quina és la complexitat de
l'algoritme anterior?

Exercici: Dijkstra

- Començant al node A:
 - dibuixa la taula de distàncies immediates a tots els nodes per a cada iteració del mètode;
 - mostra l'arbre de camins mínims.



Exercici: Bellman-Ford

- Començant al node A: dibuixa la taula de distàncies immediates a tots els nodes per a cada iteració del mètode.

